AI Trip Planner: a One-Person AI Travel Agency

ELEC5620 Model-Based Software Engineering, Stage 1 Report

Requirements documentation, architecture design and system modelling

Member	Name	SID	GitHub	Area
A	Ziqi He	530527086	`@Lilstanie`	Orchestrator and integration
B	Tingsong Jin	540532755	`@fonever2`	Itinerary and transport
C	Yi Qiao	550378747	`@HeadmasterEggy`	Accommodation and budget
D	Jiahang Bian	550066431	`@jbia0391`	Destination guide and dining
E	Weihao Wang	550414791	`@WhW0591`	Web workspace and memory

Group: Mon 10-12 Group 14

Repository: https://github.com/Lilstanie/AI_TRIP_PLANNER

Video: https://youtu.be/x9cKzPXNB1U

Contents

1. Project requirements
2. Requirement classification
3. Feature model
4. Use cases
5. Architecture analysis and design
6. Elementary structure: class model
7. Complex structure: object, collaboration and structured class
8. Behaviour models (individual)
9. Relationships between key components and future development
10. Design rationale and discarded choices
11. Change assessment and acceptance
12. Design assumptions
13. Contributions and use of generative AI

1. Project requirements

1.1 Problem

Planning a trip means visiting many sites: one for flights, one for hotels, a map for routes, blogs for food and customs, and a spreadsheet to keep the total inside a budget. Each choice constrains the others. A cheaper flight that lands late cuts the first day short, and an expensive hotel leaves no money for food. General chatbots can write an itinerary, but they invent prices and opening hours, and they never check that the parts fit together.

AI Trip Planner is a **one-person AI travel agency**. One human founder runs the agency, and a team of LLM-based agents does the work a travel agency's staff would do. A traveller describes a trip in chat. The agents then plan it together on a shared planning board, using real map, hotel, flight and weather data, and the system checks their proposals against each other and against the budget before the traveller sees the plan.

1.2 Primary users and roles

Role	Kind	Responsibility
Traveller	Human, primary user	Describes the trip, sets preferences and filters, reads and edits the plan.
Founder / operator	Human	Runs the agency: configures models, data providers and quotas, chooses mock or live data, and uses the Agent Lab to measure and compare the agents before a change ships.
AI Travel Coordinator	AI role	Talks to the traveller, turns the conversation into a structured `TripBrief`, asks clarifying questions, and explains the result.
AI Planning Supervisor	AI role	Inside the deterministic LangGraph workflow, decides which specialists a planning or revision step needs and what each should focus on.
AI Itinerary Planner	AI specialist	Builds the day-by-day schedule and checks that each day's route is feasible.
AI Transport Agent	AI specialist	Plans flights, inter-city rail and local routes, with times and fares.
AI Accommodation Agent	AI specialist	Finds and compares stays, allocates rooms for individuals or groups, and fits the stay budget.
AI Destination Guide	AI specialist	Gives grounded attractions, customs, safety, entry and packing advice, including weather.
AI Dining Agent	AI specialist	Recommends grounded venues that respect dietary needs and the meal budget.
External services	Systems	Maps and routes (Google, OpenStreetMap), hotel and flight search (SerpApi, Google Places), weather (Google, Open-Meteo), DeepSeek LLM.

The AI roles are clearly different: each has its own system prompt, inputs, tools, output schema and revision behaviour, and they collaborate through the planning board rather than one model answering everything.

1.3 Key capabilities and core features

For the traveller

- *Conversational trip intake.* The traveller writes in English or Chinese; the coordinator extracts only what was said into a `TripBrief` and asks for what is missing instead of guessing.
- *Coordinated multi-agent plan.* Five specialists plan in stages (transport, then accommodation, then itinerary and dining), each within its share of the budget left by earlier stages.
- *Conflict detection and targeted revision.* Budget overruns, schedule overlaps and impossible routes are detected deterministically and sent back only to the agents involved, for at most three rounds.
- *Grounded, labelled data.* Every price and place comes from a tool, and every section says whether its data is live, estimated, mock or a fallback.
- *Editable workspace.* The plan appears as a timeline and a map; edits re-check routes, times and the budget.
- *Memory.* Preferences and chat history persist, and signed-in travellers sync trips across devices.

For the founder / operator

- *Agent Lab.* Runs fixed scenarios through a single-agent baseline, five specialists without revision, and five specialists with targeted revision, and compares deterministic metrics.
- *Failure Lab.* Injects registered faults (provider timeout, empty result, invalid output, supervisor failure, stalled revision) to show the system degrades safely.
- *Mock or live data per request, and provider quotas,* so the agency can control cost.

1.4 Feature ownership: two core and one optional feature per member

Member	Core feature 1	Core feature 2	Optional feature
A	Conversational brief extraction and clarifying questions	Orchestration: brief validation, supervisor delegation, revision routing and round control	Agent Lab comparison and Failure Lab
B	Itinerary scheduling with route feasibility	Transport: flights, inter-city rail and local routes	Traveller-chosen leg mode and own flights
C	Accommodation for individuals or groups	Budget management with targeted savings	Keep a stay already booked
D	Destination guide: attractions, customs, safety, entry	Dining with dietary constraints	Weather-based packing advice
E	Chat workspace with timeline and map editing	Preferences and memory	Accounts and cloud sync

2. Requirement classification

2.1 Agreement, mandatory capabilities and optional features

Agreement (scope the group and the stakeholder agreed). The product is a single-user planning workspace: describe a trip, inspect grounded recommendations, edit the plan, save it and reopen it. Prices are estimates in AUD; the system does not book, pay or refund. Multi-user collaboration, social features, in-app payment and booking fulfilment are out of scope for Stage 2. Models may reason and choose, but they never decide the control flow, invent a price or place, or override a confirmed preference; deterministic code checks every model output.

Mandatory capabilities (the product is not acceptable without them):

ID	Capability	Owner
M1	Chat intake that extracts a structured trip brief and asks for missing facts	A
M2	LangGraph planning loop: dispatch, conflict detection, targeted revision (at most 3 rounds), build plan	A
M3	Day-by-day itinerary with route-feasibility checks	B
M4	Transport between and within cities, with times and fares	B
M5	Accommodation for individuals or groups within the stay allocation	C
M6	Budget roll-up in AUD, overrun spreading and infeasible-budget detection	C
M7	Grounded destination guide	D
M8	Dining recommendations that respect dietary needs	D
M9	Workspace with timeline and map views and plan editing	E
M10	Preference and session memory	E
M11	Provenance label on every section; deterministic fallback when a model or provider fails	All

Optional features (add value; the product is acceptable without them): Agent Lab and Failure Lab (A); traveller-chosen leg mode and own flights (B); keep a booked stay (C); weather-based packing advice (D); accounts and cloud sync (E); attachments in chat; place photos.

2.2 Ad hoc requirements (individual)

Informal statements in the stakeholder's own words, before any modelling.

ID	Member	Ad hoc requirement
AH-A1	A (`@Lilstanie`)	"I don't want to fill in a form. I want to type 'me and my partner, Tokyo and Kyoto, 10th to 17th November, about nine grand' and have it work out the rest. If something's actually missing, ask me once, with a few options I can click. Don't make up a budget I never gave you. While it's working I want to see what it's doing, not a loading spinner. If the plan comes back with two places an hour apart booked back to back, fix that yourself instead of handing it to me. Try a couple of times, and if it's not improving, give me the best version you got. If it can't be done on my budget, just say so."

ID	Member	Ad hoc requirement
AH-B1	B (`@fonever2`)	“I'm travelling from Sydney to Tokyo and Kyoto with a friend, on fixed dates and a shared budget. Help us choose flights and work out when to move between cities, so the daily sightseeing fits around the journey. Use fares and journey times you actually found. If I ask for a train on one leg, use it if it is offered, or tell me clearly if it isn't. If a fare is missing, say it is unknown rather than treating it as free. If the trip is too expensive or the times clash, try a suitable cheaper option or adjust the schedule; if it still cannot work, show me what needs to change.”
AH-C1	C (`@HeadmasterEggy`)	"We're five friends going to Tokyo on a fixed budget. I want the planner to work out how many rooms we need, whether we share or each get our own, and pick a hotel I'd actually stay in: decent rating, free cancellation if I asked for it. Flights get paid first, so the hotel has to fit in whatever money is left. If the whole trip ends up over budget, I want the hotel swapped for a cheaper one that still meets my rules rather than being told nothing fits. If no hotel can ever fit, tell me the minimum I'd need instead of making up a cheap one. And if I've already booked somewhere, just keep it."
AH-D1	D (`@jbia0391`)	“I’m planning a few days in Kyoto and want the guide to help me pack for the actual dates, not just guess from the season. I’m vegetarian and have a peanut allergy, so I also want restaurant ideas that take those preferences into account. Please use real place and weather evidence when it is available, tell me when a forecast is only historical climate context or unavailable, and never assure me that a restaurant is allergy-safe unless the venue confirms it.”
AH-E1	E (`@WhW0591`)	"I want to plan a trip, not just from scratch on this platform, but also with an assistant helping me organize my itinerary. I'd like to easily modify various details of the trip, while having the assistant verify for conflicts, offer suggestions, check routes and timing, and help confirm the budget. After all the creation and adjustments, I'd finally like to receive a clear and well-organized travel schedule."
ID	Type	Requirement
---	---	---
ID	Type	Requirement
ID	Type	Requirement
---	---	---
ID	Type	Requirement
ID	Type	Requirement
---	---	---

Group-level needs gathered in early lab discussion (from the team's design doc): clothing advice from the weather, accommodation for individuals or groups, food recommendations, day-by-day scheduling, luggage rules, transport, a budget, a chat window that splits a request into tasks and merges the answers, and filters for party size, area and budget.

2.3 Classified requirements

Each ad hoc requirement is broken down below. FR = functional, NFR = non-functional, C = constraint.

2.3.1 Member A (AH-A1)

ID	Type	Requirement	Traced to
R-A1	FR	Turn a free-text message into a validated `TripBrief`, merging it onto the facts earlier turns already stated.	`chat.ts: runTripChat`, `update_trip_brief`
R-A2	FR	Record only facts the traveller stated; report a missing required field instead of defaulting it.	`IncompleteBriefError`, coordinator prompt
R-A3	FR	Ask at most one clarifying question per turn, with 2-4 options written in the traveller's language.	`ask_user_question`, `ASK_USER_MAX_QUESTIONS`
R-A4	FR	Delegate to the specialists over a staged planning board, each with a concrete objective.	`supervisor.ts`, `board.ts`, `dispatch_specialists`
R-A5	FR	Stream one progress event per specialist so the traveller sees the work as it happens.	`AgentProgressEvent`, `withProgressTools`
R-A6	FR	Route a round on the detected conflicts: revise only the targeted sections, and only while a round remains.	`workflow.ts: routeAfterDetection`, `reviseConflicts`
R-A7	FR	Keep a revision round only if `planScore` improves; otherwise discard it and keep the earlier proposals.	`planScore`, `stalled`, `routeAfterRevision`
R-A8	FR	Assemble the plan from validated proposals and mark each section `draft` or `needs_you`.	`build_plan`, `toSection`
R-A9	C	At most three revision rounds per turn.	`DEFAULT_MAX_ROUNDS`
R-A10	NFR (integrity)	A model-proposed brief change is re-validated through the Zod contract before planning starts. The reply text is never treated as the plan.	`BriefPatchSchema`, `applyBriefPatch`
R-A11	NFR (availability)	With no model key, or an off-schema answer, extraction, delegation and the reply each fall back to deterministic code and the turn still completes.	`fallbackReplyFor`, supervisor fallback
R-A12	NFR (verifiability)	Orchestrator behaviour under injected model and provider faults is replayable and scored, so these rules are checked under failure conditions.	`orchestrator/src/agent-lab/`

2.3.2 Member B (AH-B1)

AH-B1 is a coursework stakeholder-style statement, not an interview quotation. Its six classified requirements trace to the current transport and itinerary implementation.

ID	Type	Requirement	Current implementation evidence
R-B1	FR	Derive ordered journey hops from origin, destinations and dates; gather flight and ground-route options and produce a timed transport proposal for the party.	packages/agents/src/transport/index.ts:87; `journeyLegs` in `transport/legs.ts`

ID	Type	Requirement	Current implementation evidence
R-B2	FR	Use the transport schedule when planning daily activities and check that travel between different activity locations fits the available gap, including a 15-minute arrival buffer.	packages/orchestrator/src/board.ts:15; `citiesByDay`, `travelConflicts`: packages/agents/src/itinerary/index.ts:342
R-B3	FR	Apply a traveller's per-hop mode choice when supported; explicitly report an unavailable choice and the route used instead.	`chosenModeFor`: packages/agents/src/transport/legs.ts:106; `scheduledLegs` and `layOutHop`: packages/agents/src/transport/index.ts:306
R-B4	NFR — integrity	Select only returned fare candidates; derive costs and durations from evidence. Preserve absent ground fares as unpriced and report unavailable required flights without inventing a price.	`planTransport`: packages/agents/src/transport/index.ts:685; `assembleTransportProposal`: packages/agents/src/transport/index.ts:470
R-B5	NFR — resilience	When no model is configured or its selection is invalid, use a deterministic evidence-based plan; retain availability gaps and source labels. Unrecoverable input/provider errors may still fail the run.	`deterministicPlan`, `planTransport` catch path: packages/agents/src/transport/index.ts:685
R-B6	C — bounded control	Under the default workflow, allow at most three total planning rounds (initial round plus at most two revision rounds); rerun only targeted revisable specialists and retain a revision only if the plan score improves.	packages/orchestrator/src/workflow.ts:141; packages/orchestrator/src/conflicts.ts:24

2.3.3 Member C (AH-C1)

ID	Type	Requirement	Traced to
R-C1	FR	Compute the room count from party size and room allocation: `individual` gives one room per guest, `shared` gives ⌈guests / 2⌉.	`accommodation/index.ts`
R-C2	FR	Filter stay candidates by minimum rating and, when requested, free cancellation; these are hard rules that a budget revision may not relax.	`planning.ts: eligibleOptions`
R-C3	FR	Choose the first stay preferring rating ≥ 8 and free cancellation, within the stay allocation left after transport.	`chooseInitial`, board allocation
R-C4	FR	Roll up section costs in AUD and compare the total with `budgetTotal`.	`budget.ts: rollUpCost`
R-C5	FR	On any overrun, spread the required saving over the sections that can still be cut, and send each a targeted revision request.	`conflicts.ts: detectConflicts`
R-C6	FR	When the cheapest options already exceed the budget, report one `infeasible budget` conflict naming the minimum, and stop revising.	`minimumCost`, `INFEASIBLE_BUDGET`
R-C7	FR	Keep a stay the traveller has already booked, unpriced and without a search.	`bookedStayProposal`
R-C8	NFR (accuracy)	Money is summed in integer cents so totals never drift.	`sumMoney`, `stayCost`

ID	Type	Requirement	Traced to
R-C9	NFR (integrity)	The model may choose only among searched candidate ids; it never invents a property, rate or policy.	accommodation system prompt
R-C10	NFR (availability)	With no model key, or an off-schema answer, a deterministic fallback still produces a valid proposal.	`planStays` fallback
R-C11	C	At most three revision rounds; a round is kept only if the plan score improves.	`workflow.ts`

2.3.4 Member D (AH-D1)

ID	Type	Requirement	Traced to
R-D1	FR	For a valid trip date and destination coordinates, request weather evidence and include weather-based packing guidance in the destination-guide proposal.	`destination-guide/index.ts:planDestinationGuide`
R-D2	NFR (accuracy)	Label weather by its actual horizon: forecasts cover up to 14 days; later dates use climate context, never a forecast. Preserve provider, observation time and forecast validity where supplied.	`weather.ts:weather.forecast`; `destination-guide/index.ts:planDestinationGuide`
R-D3	FR	Use grounded restaurant candidates and relevant stored dietary preferences to produce dining suggestions within the meal-budget ceiling.	`dining/index.ts:dietaryPreferences`; `dining/index.ts:budgetCeiling`
R-D4	NFR (integrity)	Accept attraction and restaurant names only when they match map candidates; do not claim menus, allergen safety or dietary suitability as verified facts.	`destination-guide/index.ts:validateDraft`; `dining/index.ts:validateDraft`; `dining/index.ts:createMiniMaxGenerator`
R-D5	NFR (availability)	If weather or model evidence is unavailable or invalid, return deterministic guidance where possible and label unavailable weather or fallback provenance rather than presenting it as a live forecast.	`destination-guide/index.ts:fallbackDraft`; `destination-guide/index.ts:planDestinationGuide`

2.3.5 Member E (AH-E1)

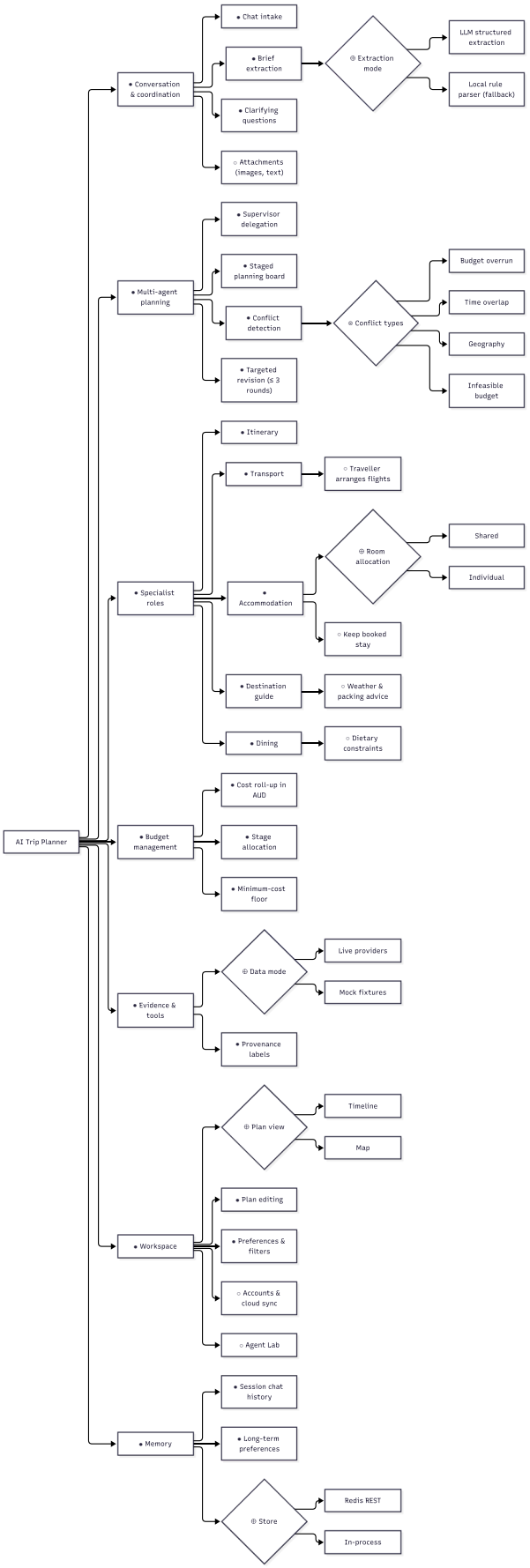
ID	Type	Requirement	Traced to
R-E1	FR	Accept timeline/map edits, including moving, retiming, replacing, route checking and undo.	`EditRequest`; `TripEditor`; `useTimelineEdits`
R-E2	FR	Show a preview before applying, including differences, routes, blockers, conflicts, costs and budget status.	`EditPreview`; `EditPreviewPanel`; `previewEdit`
R-E3	FR	Preserve unchanged activities and ideas, share selection between timeline and map, and flag unverified prices.	`TripPlan.editIssues`; shared selected activity id

ID	Type	Requirement	Traced to
R-E4	NFR	Recalculate routes, schedule conflicts, section conflicts, costs and budget deterministically after edits.	`detectConflicts`; `rollUpCost`; Maps / Routes API
R-E5	NFR	Keep trip facts separate from preferences. The current chat LLM updates briefs or replans; it does not call the edit boundary.	`TripBrief`; coordinator tools; memory service
R-E6	C / NFR	Enforce destination-segment rules and reject malformed or stale previews without inventing route, price or place.	`baseVersion`; Zod; identity and segment checks

3. Feature model

3.1 Feature diagram (group)

Notation: ● mandatory, ○ optional, ⊕ alternative (exactly one), ⊗ or (one or more).



3.2 Cross-tree constraints

1. *Targeted revision* requires *Conflict detection*.

2. *Budget overrun* requires *Cost roll-up in AUD*; *Infeasible budget* requires *Minimum-cost floor*.
3. *Keep booked stay* excludes searching and pricing in *Accommodation* for that trip.
4. *Traveller arranges flights* excludes fare search in *Transport* and any flight-fare conflict.
5. *Live providers* requires provider keys; without them the system selects *Mock fixtures*.
6. *Accounts & cloud sync* requires *Long-term preferences*.

3.3 Non-functional requirements attached to features

Feature	NFR
Multi-agent planning	Every proposal is re-validated against the shared Zod schema at each graph boundary.
Specialist roles	Each specialist falls back to deterministic output when the model is missing or off-schema, so a request always completes.
Budget management	AUD arithmetic in integer cents; overrun percentages are compared unrounded.
Evidence & tools	Prices and places come only from tools; every section shows whether its data is live, estimated, mock or fallback.
Workspace	Planning progress streams as NDJSON so the traveller sees each stage as it runs.
Agent Lab	Fixture runs can never reach a paid model or provider.

4. Use cases

4.1 Overall use case diagram

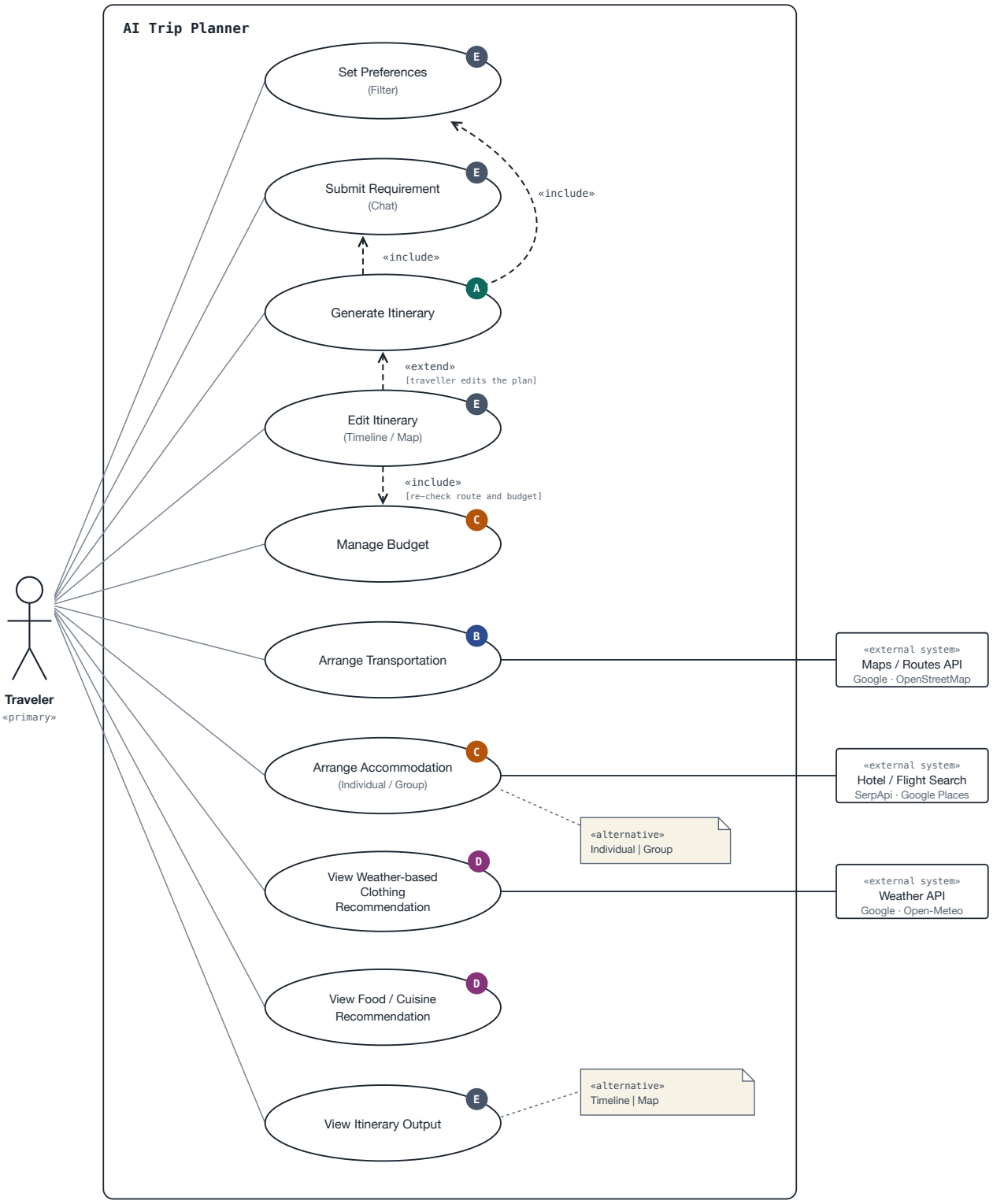


Figure 4.1 Overall use case diagram. Actors: **Traveler** (primary), and three external systems: **Maps / Routes API** (Google, OpenStreetMap), **Hotel / Flight Search** (SerpApi, Google Places) and **Weather API**

(Google, Open-Meteo).

Use case	Owner	Relationships
Set Preferences (Filter)	E	—
Submit Requirement (Chat)	E	—
Generate Itinerary	A	«include» Arrange Transportation, Arrange Accommodation, Manage Budget
Edit Itinerary (Timeline / Map)	E	«extend» Generate Itinerary [traveller edits the plan]
Arrange Transportation	B	«include» Maps / Routes API
Arrange Accommodation (Individual / Group)	C	«include» Hotel / Flight Search
Manage Budget	C	«extend» Edit Itinerary [re-check route and budget]
View Weather-based Clothing Recommendation	D	«include» Weather API
View Food / Cuisine Recommendation	D	—
View Itinerary Output	E	«alternative» Timeline / Map

4.2 Use case specifications

One specification per member at least (C wrote two); seven in total. Each comes from that member's ad hoc requirement in §2.2.

UC-A1 Generate Itinerary

Field	Content
ID / name	UC-A1 Generate Itinerary
Source	AH-A1; R-A1 … R-A12
Primary actor	Traveler
Secondary actors	LLM (DeepSeek) as coordinator, supervisor and reply writer; the five specialist agents; Maps / Routes, Hotel / Flight Search and Weather APIs through the specialists
Goal	Turn a free-text message into a validated, internally consistent `TripPlan`, or state what is missing or why the trip cannot be planned.
Trigger	The traveller sends a chat message to `POST /api/chat`.
Preconditions	The message is non-empty or carries an attachment. Nothing else is required, because resolving the missing facts is part of this use case.
Postconditions (success)	A `TripPlan` exists with five sections, each `draft` or `needs_you`, `estTotal` and `overrunPct` computed, and a reply naming what the traveller must still decide. Every costed item comes from a specialist's evidence.
Postconditions (partial)	A `needs_info` or `ask_user` frame is returned with the facts understood so far, and no plan is produced.
Postconditions (failure)	An error frame naming the specialist that failed. No partial plan is returned.

Main success scenario

1. Coordinator merges the message onto the facts earlier turns stated (`known`).

2. The LLM reads the merged context and calls `update_trip_brief` with only the facts this message states.
3. Coordinator validates the patch through `BriefPatchSchema`, applies it with `applyBriefPatch`, and parses the result as a `TripBrief`; the dates must be real and ordered, with at least one night per city.
4. Coordinator starts the LangGraph workflow with the validated brief.
5. `dispatch_specialists` asks the supervisor which specialists to call; the supervisor writes one concrete objective per specialist, naming the trip facts it must respect.
6. Each specialist plans its own section and posts an `AgentProposal` on the staged planning board; one progress event per specialist reaches the browser as it works.
7. `detect_conflicts` rolls up the costs and collects every proposal's conflicts (member C's `detectConflicts`).
8. No conflicts remain: `build_plan` assembles the plan, marking each section `draft`.
9. Coordinator builds a digest of the plan, the LLM writes the reply from it, and the final frame carries both reply and plan.

Extensions

- 2a. *No model configured, or off-schema output*: no patch is applied; the turn proceeds on the facts already stated, and the reply is written by `fallbackReplyFor` (R-A11).
- 3a. *A required fact is still missing* (destination, dates, group size or budget): the turn ends with `IncompleteBriefError` naming the missing fields, carrying `known` so the traveller does not repeat the rest. No default values are used (R-A2).
- 3b. *Dates are unreal, reversed, or shorter than one night per city*: the brief is rejected with the reason; planning does not start.
- 3c. *The traveller states a lasting wish, a travel mode or a booked stay*: it is recorded on the brief (`learnedPreferences`, `legModes`, `bookedStay`) and the plan is re-run with it.
- 4a. *The traveller asked a question the plan already answers*: the coordinator answers from the plan and does not replan.
- 5a. *The coordinator would rather ask than guess*: it calls `ask_user_question` with 2-4 options in the traveller's language, recommended option first. The turn ends there, and no other tool runs after it (R-A3).
- 5b. *Supervisor unavailable or off-schema*: all five specialists are dispatched deterministically, and the itinerary specialist is never skipped (R-A11).
- 6a. *A specialist throws*: the run stops and the error frame names it. No partial plan is returned.
- 7a. *Conflicts remain, a round is left, and none is `infeasible budget`*: `revise_conflicts` re-invokes only the targeted specialists, each with its previous proposal and any `targetSaving`, then returns to step 7 (R-A6).
- 7b. *`planScore` did not improve after a revision*: the round is discarded, the earlier proposals are kept, `stalled` is set, and the flow goes to step 8 (R-A7).
- 7c. *Three rounds have run and conflicts remain*: the plan is built with each still-targeted section marked `needs_you` and `conflicts` listing what is unresolved (R-A9).
- 7d. *`infeasible budget`*: the revision loop is skipped entirely and the reply names the minimum budget needed (member C's UC-C2, extension 2b).

Special requirements: R-A10 (a model-proposed change is re-validated before planning starts), R-A11 (every LLM step has a deterministic fallback), R-A12 (the behaviour above is replayed under injected faults in `agent-lab`).

UC-B1 Arrange Transportation

Field	Content
ID / name	UC-B1 Arrange Transportation
Source	AH-B1; R-B1–R-B6
Primary actor	Traveler
Secondary external actors	Flight search provider; Maps / Routes providers; model service when configured
Internal collaborating roles	Workflow / supervisor, Transport specialist, planning board, Itinerary specialist, conflict policy. These are internal system roles, not additional human actors.
Goal	Obtain evidence-based transport recommendations coordinated with the daily itinerary, with known prices and any unresolved availability, timing or budget problems made explicit.
Trigger	The traveler requests a trip plan or a transport-related change; the workflow dispatches Transport, or routes a targeted revision to it.
Preconditions	A schema-valid TripBrief exists with valid ordered ISO dates, destination, positive group size and budget. Maps, Booking and Memory ports are injected. A usable model is optional. Normal scenario supplies an explicit origin.
Success postconditions	A schema-valid transport proposal contains provider-based fare choices and scheduled ground legs, is available to downstream planning, and is incorporated into TripPlan. No unresolved request targets transport. Relevant activities have been checked against route evidence. No booking or payment is performed.
Partial postconditions	The best retained proposal preserves known prices, unpriced warnings, unavailable choices and/or conflicts. Transport is `needs_you` only while an unresolved request targets it; otherwise `draft`, including when a ground fare is unknown.
Failure postconditions	Invalid input, cancellation or an unrecoverable error stops the run; no successful plan is claimed. Invalid model output alone normally falls back rather than failing.

Main success scenario

1. The traveler supplies origin, destinations, dates, party size, budget and any per-leg mode choice. Chat intake produces a validated brief upstream of this use case.
2. The workflow dispatches `transportAgent.invoke` through its specialist boundary and planning board, with injected tools and memory.
3. Transport validates the dates, derives the ordered journey and reads any relevant origin preference. It gathers offered flight fares and ground routes, plus route alternatives when the port supports them.
4. With a configured model, the Transport LLM calls `search_transport_evidence`, then selects offered flight IDs and a planning day/local departure time for each routed hop. It does not supply prices or durations.
5. Deterministic code resolves selected IDs against the evidence, requires exactly one fare per hop with offered fares, checks required hop coverage, planning-day bounds and HH:mm syntax. It checks route validity and whether each scheduled hop fits within its planning day.
6. Code assembles the proposal from provider prices and durations, applies supported mode preferences, and records assumptions, a cost floor, available flight alternatives and a source label. If an allocation is exceeded, the chosen flights give way to the cheapest returned fares.
7. The workflow validates the proposal schema; the planning board stores it. In the staged first round, itinerary planning can read transport and accommodation proposals when those dependencies

were dispatched. It grounds activities, validates its draft and checks connections using route duration plus a 15-minute buffer.
8. Once dispatched proposals are collected, deterministic conflict policy checks known budget totals, reported route problems and scheduled-item overlaps.
9. With no unresolved conflict, the workflow assembles the plan and returns transport as `draft`, together with the known estimate and source information. The traveler can inspect or change it in chat/editor.

Extensions

- **1a — Invalid input:** schema/date validation rejects an invalid brief or unordered dates. Correct the input; do not claim a valid plan.
- **3a — Origin omitted:** resolution uses the brief origin, then `transport.origin` memory, then the first destination. When the first destination is used, no long-haul arrival flight is planned. The proposal discloses the resolved origin; this fallback is not a user-confirmation step.
- **3b — Traveler arranges flights:** `excludeFlights` skips flight searches/pricing and does not raise a missing-fare conflict for those self-arranged flights.
- **3c — Required flight unavailable or empty:** retain a missing-flight conflict and an incomplete estimate; never fabricate a fallback fare.
- **3d — No usable ground route or route cannot fit:** omit the invalid scheduled hop and record a geography/time conflict. For an unchosen, overlong inter-city ground hop, evidence gathering may first seek a flight instead; an explicit ground-mode choice is not silently overridden for this reason.
- **4a/5a — Model absent, fails or returns an invalid selection:** use a deterministic plan from gathered evidence (gather it if needed). Model-error fallback is labelled `Local fallback`; the no-model path uses normal evidence provenance. Fallback cannot create missing provider evidence.
- **6a — Ground fare unavailable:** omit `estCost`, count the unpriced leg and describe the known estimate as a lower bound. This alone does not trigger a conflict or a revision.
- **6b — Chosen ground mode unavailable:** retain the provider's route and explicitly state the requested mode was unavailable. This alone is a warning, not a revision request.
- **8a — Feasible conflict:** invoke only the targeted revisable specialist(s), passing previous proposals, revision constraints and an allocation where applicable. Repeat evidence gathering, selection and validation. Activity overlaps target itinerary so it moves around transport rather than forcing both to move. Keep a round only if `planScore` strictly improves, then recheck.
- **8b — Infeasible budget:** stop revision and report the evidence-based minimum. The conflict targets the largest-cost section; transport is not necessarily that section.
- **8c — Default third round reached or score does not improve:** return the best retained plan; mark only still-targeted sections `needs_you`. A non-improving round is discarded.
- **Any step — Cancellation/unrecoverable failure:** stop, report the failure and allow a later retry. A future chat/edit starts a new interaction; there is no `confirmed` or `booked` state in this use case.

Special requirements: R-B4–R-B6. Trip dates use the current end-exclusive planning interval. Transport estimates are in AUD and are not reservations. Ground-provider durations are gathered before model scheduling; changing the chosen departure time does not automatically re-query time-sensitive transit schedules. Availability gaps must remain visible; `draft` is not a guarantee of complete pricing.

UC-C1 Arrange Accommodation

Field	Content
ID / name	UC-C1 Arrange Accommodation (Individual / Group)

Field	Content
Source	AH-C1; R-C1, R-C2, R-C3, R-C7, R-C9, R-C10
Primary actor	Traveler
Secondary actors	Hotel / Flight Search (SerpApi Google Hotels, Google Places), Orchestrator (system role)
Goal	Every night of the trip has a stay that respects the traveller's room and quality rules and fits the money left after transport.
Trigger	The Orchestrator's `dispatch_specialists` node delegates to the Accommodation agent once Transport has finished on the planning board.
Preconditions	1. A valid `TripBrief` exists (destination, dates with at least one night per city, `groupSize` ≥ 1, `budgetTotal`). 2. Transport's proposal is on the board, so a stay allocation can be computed.
Postconditions (success)	An `AgentProposal` for `accommodation` is on the board with one `StaySelection` per city, its cost, a `floorCost` (cheapest eligible total) and a provenance label.
Postconditions (failure)	No proposal; the run stops with the accommodation specialist named as the failure. No stay is invented.

Main success scenario

1. Orchestrator invokes the Accommodation agent with the brief, the board and a stay allocation (40% of the budget left after transport).
2. Agent reads the traveller's lodging preferences: room allocation, minimum rating, free cancellation.
3. Agent computes the room count: one per guest for *individual*, ⌈guests / 2⌉ for *shared*.
4. Agent splits the trip into stay segments per city, using the inter-city hops Transport scheduled.
5. For each segment, agent searches the booking port for stays in that city and dates.
6. Agent removes malformed and preference-incompatible candidates and sorts the rest cheapest first.
7. The model chooses one candidate id per segment, weighing rating, cancellation and total cost against the allocation.
8. Agent validates the choice against the candidate list and prices it as rooms × nights × nightly rate, in cents.
9. Agent records the cheapest eligible total as `floorCost` and returns the proposal to the board.
10. Orchestrator includes the accommodation section in conflict detection (UC-C2).

Extensions

- 2a. *Traveller has already booked a stay* (`bookedStay` set): agent returns that stay unpriced with `floorCost` 0 and skips steps 3–9.
- 4a. *Transport scheduled no usable hops*: nights are split evenly across cities, extra nights to earlier cities.
- 5a. *Live search unavailable*: Google Places estimates are used and the proposal is labelled "estimated", naming the provider that failed.
- 6a. *No candidate survives the filters*: the agent raises an error and the run stops; it does not relax a confirmed preference.
- 7a. *No model configured, or the model returns an unknown id or off-schema output*: the deterministic rule picks the cheapest candidate with rating ≥ 8 and free cancellation (or the cheapest) within the allocation, labelled "Local fallback".
- 7b. *First choice exceeds the allocation*: the best stay within it is taken, or the cheapest if none fit, and the proposal says why.
- *Any step*. *Revision request received* (from UC-C2): see UC-C2 step 6.

Special requirements: R-C8 (cent arithmetic), R-C9 (no invented properties), R-C10 (fallback).

UC-C2 Manage Budget

Field	Content
ID / name	UC-C2 Manage Budget
Source	AH-C1; R-C4, R-C5, R-C6, R-C11
Primary actor	Traveler
Secondary actors	Orchestrator, the five specialist agents
Goal	The plan the traveller receives fits the budget, or says plainly by how much it cannot.
Trigger	All dispatched specialists have put a proposal on the board (`detect_conflicts`), or the traveller edits the plan.
Preconditions	`budgetTotal` ≥ AUD 0.01; at least one proposal exists.
Postconditions (success)	`TripPlan.estTotal` ≤ `budgetTotal`; no budget conflict remains.
Postconditions (partial)	The plan is returned with the best total found; each section still targeted is marked `needs_you`, and `conflicts` lists what remains.

Main success scenario

1. Orchestrator rolls up every proposal's item costs in AUD cents and computes `overrunPct`.
2. Total is within budget: no budget conflict is raised.
3. Orchestrator builds the plan, marks each section `draft` and returns it with `estTotal` and `overrunPct`.

Extensions

- 2a. *Total over budget, but the sum of floor costs fits*:
    1. Orchestrator computes each section's room to cut (its cost less its `floorCost`).
    2. It spreads the overrun over those sections in proportion, creating one `RevisionRequest` per section with a `targetSaving`.
    3. Orchestrator runs `revise_conflicts` for those agents only, giving each its previous proposal and a new allocation (last cost less saving).
    4. Accommodation, on a budget revision, keeps the confirmed preferences and picks the cheapest eligible stay within the new allocation.
    5. Orchestrator re-runs detection and computes the plan score (AUD over budget + 10% of budget per other conflict).
    6. Score improved: keep the round and go back to step 1 of the main scenario. Score not improved: discard the round and go to 2c.
- 2b. *Sum of floor costs already exceeds the budget*: Orchestrator raises one `infeasible budget` conflict naming the minimum, stops revising, and the reply tells the traveller to raise the budget to at least that amount or change dates, origin or destination.
- 2c. *Three rounds reached, or no improvement*: the best plan so far is returned; sections still targeted are `needs_you`.
- *Traveller edits the plan*: costs are re-checked in the editor and a `price_unverified` issue is shown for any unpriced change.

Special requirements: R-C8; overrun percentages are compared unrounded.

UC-D1 View Weather-based Clothing Recommendation

Field	Content
ID / name	UC-D1 View Weather-based Clothing Recommendation
Source	AH-D1; R-D1–R-D5
Primary actor	Traveler
Secondary actors	Orchestrator (system role), Maps / Places API, Weather API (Google Weather API or Open-Meteo), long-term preference store
Goal	The traveller receives destination and weather-based packing guidance plus grounded dining candidates informed by confirmed dietary preferences, with evidence limits made clear.
Trigger	The traveller submits a trip request and the Orchestrator dispatches the specialist agents for the valid `TripBrief`.
Preconditions	1. A valid `TripBrief` exists with destination, travel dates, group size and user id. 2. The MapsPort, WeatherPort and memory port are injected; coordinates and saved dietary preferences may be absent.
Postconditions (success)	Destination-guide and dining `AgentProposal`s are included in the assembled `TripPlan`; attractions and venues match map candidates, and weather horizon/source are stated.
Postconditions (partial)	If coordinates, forecast data or a model are unavailable, the plan still includes available deterministic guidance and labels weather as climate context or unavailable rather than a forecast.
Postconditions (failure)	If required map or memory retrieval fails, the affected specialist fails and the workflow reports the specialist failure; it does not invent place evidence.

Main success scenario

1. The traveller submits destination, dates, party size and dietary requirements such as vegetarian food and a peanut allergy.
2. The Orchestrator dispatches Destination Guide and Dining with the validated `TripBrief` and injected maps, weather and memory ports.
3. Destination Guide validates the trip dates, then retrieves sight and museum candidates and saved preferences.
4. It removes duplicate map candidates and uses a candidate coordinate to request weather for the first travel date.
5. The weather adapter selects a forecast or climate source for the date and returns the horizon and provenance with the result.
6. The configured LLM reads the validated brief, month, place candidates and preferences through its read-only evidence tool, then drafts destination and packing guidance.
7. Deterministic code fits the draft to the output schema, rejects ungrounded attraction names and deduplicates accepted candidates; invalid output uses the local fallback.
8. Dining retrieves restaurant candidates and saved preferences, filters to dietary-related preferences and computes a per-person meal ceiling.
9. The configured LLM drafts venue suggestions from its evidence tool. Deterministic code checks the budget and candidate names, while the proposal says menus and allergy suitability must be confirmed with the venue.
10. The Orchestrator validates the proposals at the graph boundary, assembles the `TripPlan` and returns the weather-packing and dining sections to the traveller.

Extensions

- 4a. *No candidate has coordinates or no WeatherPort is available*: skip the forecast request and use month-level planning context; do not present it as a forecast.
- 5a. *The trip date is more than 14 days away*: return historical climate context, explicitly not a forecast. In mock mode, the weather fixture follows the same horizon distinction.
- 5b. *Weather provider fails*: continue with month-level context and label the weather provider unavailable; do not fail the destination guide solely because weather failed.
- 7a. *No model is configured, the model fails, or its draft is invalid or names an ungrounded attraction*: return deterministic guidance from available map evidence and mark the proposal as fallback.
- 8a. *No restaurant candidates are returned*: return the meal-budget envelope without venue picks.
- 9a. *No saved dietary preferences exist*: generate general grounded candidates and state that no confirmed dietary preferences were found.
- 9b. *A traveller reports an allergy*: never guarantee a venue is safe; tell the traveller to confirm ingredients and cross-contamination controls directly with the venue.

Special requirements: R-D2 (forecast horizon and provenance), R-D4 (grounded names and dietary safety), R-D5 (visible degradation and fallback).

UC-E1 Edit Itinerary (Timeline / Map)

Field	Content
ID / name	UC-E1 Edit Itinerary (Timeline / Map)
Source	AH-E1; R-E1–R-E6
Primary actor	Traveler
Secondary actors	Web workspace, deterministic itinerary editor, Maps / Routes / Places API, browser local storage; the chat LLM is a separate path and is not invoked by this use case
Goal	Change a scheduled activity while preserving a valid, explainable and current trip plan.
Trigger	The traveller selects or drags a stop in the timeline/map, changes its time/place, requests a route check, or chooses undo.
Preconditions	A `TripPlan` is open; it has an itinerary proposal; activities have unique IDs and complete schedule data; the client holds the current `editVersion`.
Postconditions (success)	The validated preview is applied to client state and autosaved in browser storage; routes, conflicts, costs and edit issues are shown.
Postconditions (blocked / failure)	The current client plan remains unchanged and the preview reports an invalid operation, unavailable route, unverified price or stale preview.

Main success scenario

1. The traveller selects a stop in the timeline or map, changes its time/place, requests a route check, or chooses undo.
2. The workspace identifies the activity by stable ID and builds a typed `EditRequest`.
3. The workspace sends the plan, `baseVersion`, operation and route mode to `/api/trip/preview-edit`.
4. The deterministic editor parses the request, checks plan identity and rejects a stale `baseVersion`.
5. The editor applies the operation to a copy and checks day, time, ID and destination-segment rules.
6. For affected days, Maps / Routes is queried for travel time between consecutive places.
7. The editor recomputes route blockers, conflicts, section costs, total cost and budget status.

8. A changed place receives a `price_unverified` issue; route verification does not currently clear that price issue.
9. The workspace shows a preview with differences, route results, blockers and budget consequences.
10. The client compares `preview.baseVersion` with the current plan version, then applies the plan to React state and autosaves it to localStorage.

Extensions

- *Chat path*: the coordinator may update the trip brief, replan, answer a question, or return the existing plan. It does not create an `EditRequest` for this use case.
- 4a. *Stale plan at preview time*: return "Start from the current plan"; keep the current client plan unchanged.
- 5a. *Move crosses destination accommodation segments*: reject the move because the destination stay would no longer match the itinerary.
- 6a. *End time is not after start time, the day is outside the trip, or IDs are not unique*: return a blocker and do not apply.
- 7a. *A place is missing*: ask the traveller to confirm/search for the place before checking routes.
- 7b. *The route provider fails*: show `route_unavailable`; preserve the preview but do not claim the route is valid.
- 8a. *The edit causes a schedule or budget conflict*: show the conflict and mark the affected section `needs_you`.
- 9a. *A place change affects cost*: show `price_unverified`; the current verify operation rechecks routes but does not implement separate price clearance.
- 11a. *The traveller cancels the preview*: discard the preview and keep the old plan.
- 12a. *Undo is requested*: restore the saved activity snapshot, rerun validation and create another plan version.

LLM versus deterministic control

Concern	Current LLM / chat behaviour	Current deterministic responsibility
Chat request	Updates the brief, replans, asks a question or answers; no `EditRequest` tool	N/A for this edit boundary
Timeline/map edit	Not invoked	Parse and validate `EditRequest`, IDs, dates, times and segments
Route and budget	Not invoked	Query providers; compute routes, blockers, conflicts, costs and status
Price uncertainty	May explain the result	Set `price_unverified`; no complete price-clearance operation yet
Memory	Drops learned preferences when the setting is off	Long-term memory service stores/reads preferences; specialist read gating is not fully tied to the setting
Apply changes	No direct write access to the edit endpoint	Client version check, React state update and localStorage autosave

Special requirements: R-E4 (deterministic recalculation), R-E6 (stale or malformed previews are rejected without inventing a route, price or place).

5. Architecture analysis and design

5.1 Viewpoints

The architecture is described from four viewpoints, chosen for the concerns of each stakeholder.

Viewpoint	Stakeholder and concern	Where it is modelled
Logical	Developers: which classes and contracts exist, and how they relate	Class model (§6), feature model (§3)
Process	Founder and developers: how agents run, wait for each other, retry and stop	Ad hoc design diagram (§5.2), behaviour models (§8), collaboration (§7.2)
Development	The five members: who owns which package, and which dependencies are allowed	Package diagram (§5.3)
Physical	Founder: where the system runs and which external services it pays for	Deployment diagram (§5.4)
Scenarios	Traveller: what the system does for them	Use cases (§4)

5.2 Ad hoc design diagram: runtime flow

One request from the traveller's message to a validated plan. Solid arrows are the normal path; dashed arrows are fallbacks when a model or provider is unavailable.

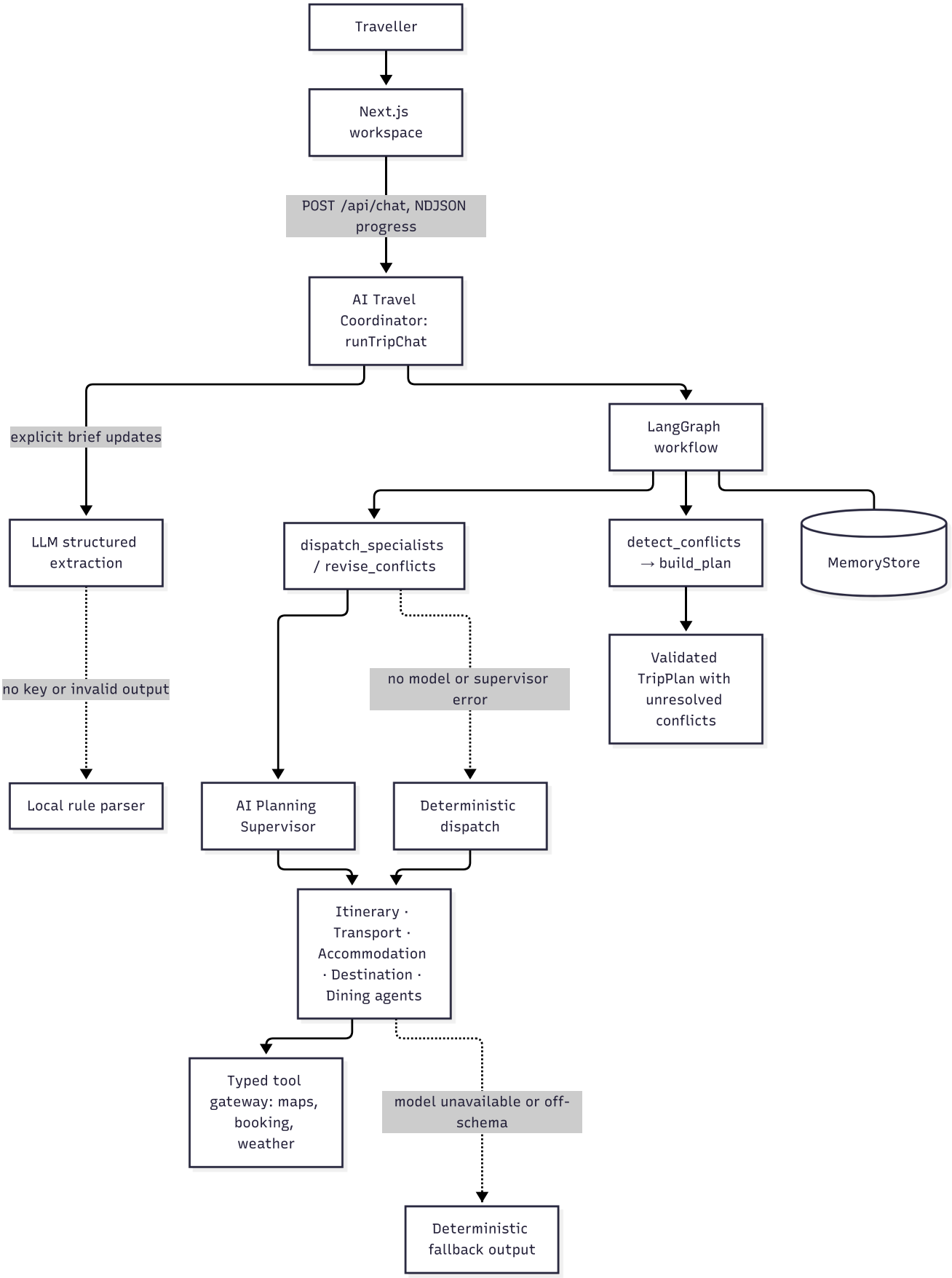


Figure 5.1. Runtime flow.

The LangGraph workflow drives the supervisor, not the other way round. The graph decides when to dispatch, revise and stop; the supervisor only chooses which specialists a step needs. The control path,

budget red lines and stopping condition never depend on a model improvising the next step.

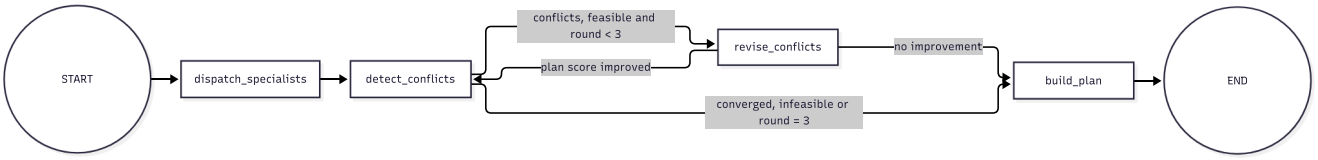


Figure 5.2. The LangGraph planning loop.

5.3 Package diagram

The repository is a pnpm and Turborepo monorepo. Dependencies point inward to the shared contracts; `shared` depends on nothing, and no package imports the web app.

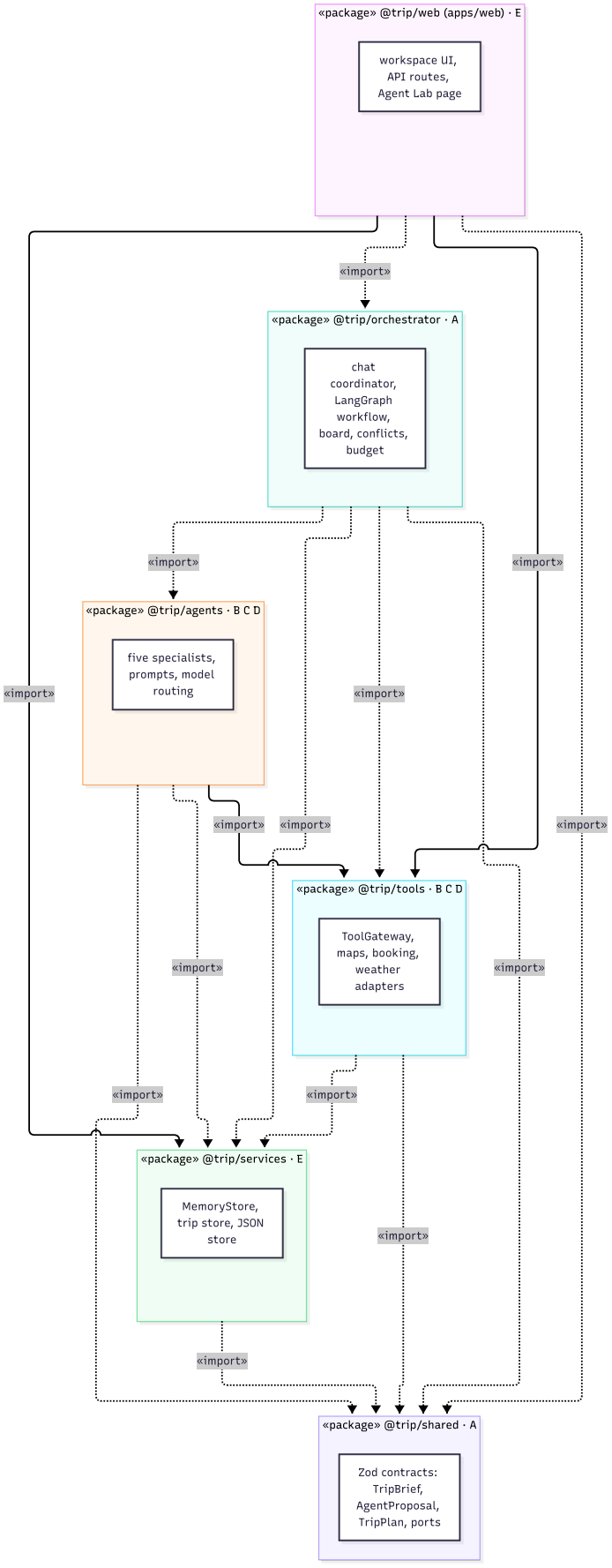


Figure 5.3. Package diagram, with the owning member of each package.

5.4 Deployment diagram

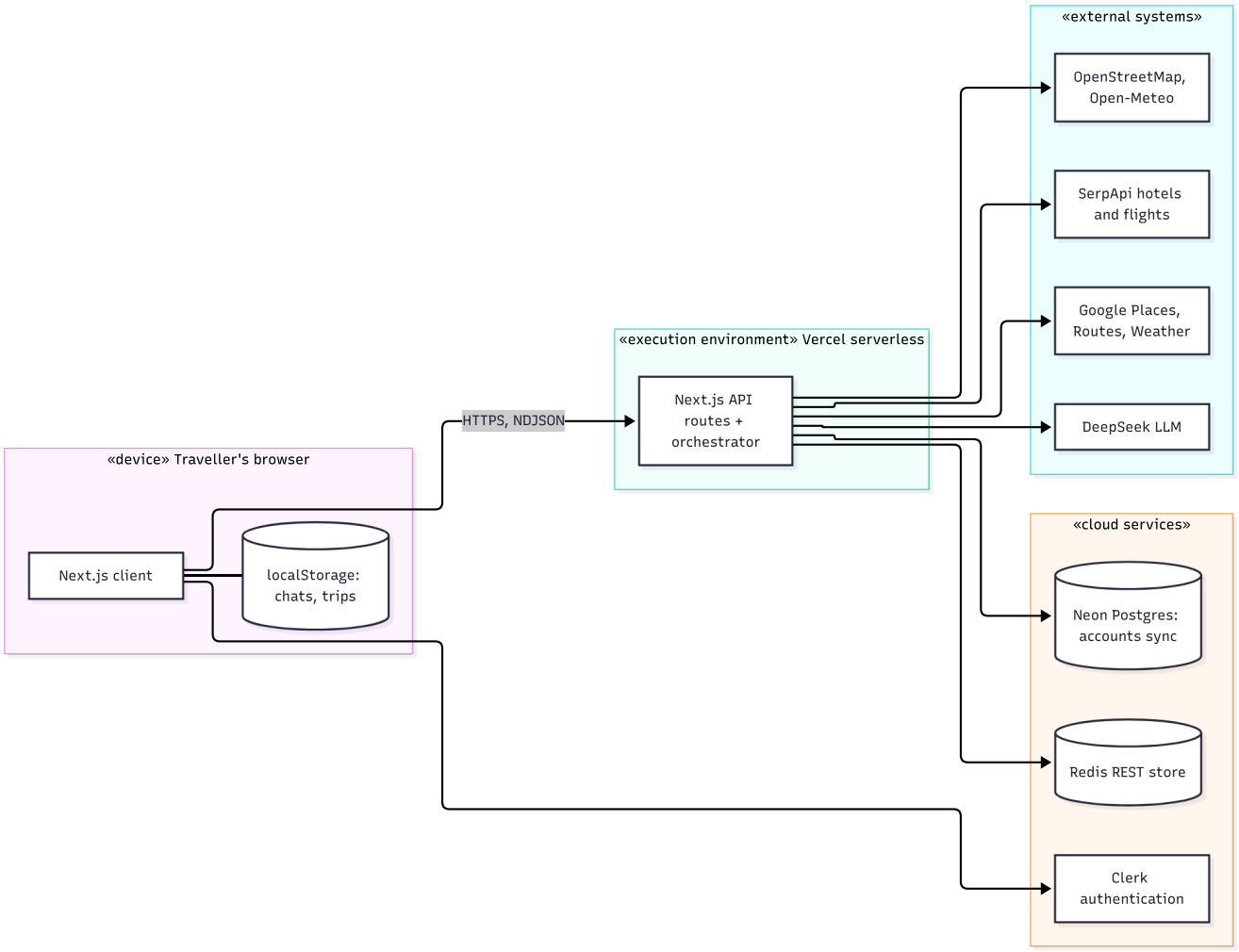


Figure 5.4. Deployment diagram.

Without provider keys the same build runs fully offline on mock fixtures and process memory, which is how the Agent Lab and the tests run.

5.5 Life-cycle process

The group follows an iterative, agile process. Work is tracked as GitHub issues with triage labels and will be mirrored in Jira for Stage 2, as the brief requires. Each change is a short-lived branch merged into `main` through a pull request after CI (typecheck, lint, tests, build, documentation and protected-file checks). Decisions with lasting rationale are recorded as Agent Notes in the repository, so the design model and the code are reviewed in the same pull request.

6. Elementary structure: class model

The class model has six UML class diagrams that share one namespace, drawn from the code. Modules that export functions appear as classes with the `«module»` stereotype. Diagrams 1 to 5 show composition, aggregation, association, dependency, multiplicity, interfaces and realisation; Diagram 6 adds generalisation.

6.1 Notation

Mark	Relationship	Meaning
solid line, hollow triangle	generalisation	subclass → superclass ("is a kind of")
dashed line, hollow triangle	realisation	class → interface ("implements")
solid line, filled diamond	composition	whole ◆ part; part cannot outlive the whole
solid line, hollow diamond	aggregation	whole ◇ part, shared; part has an independent lifetime
solid line, open arrow	association	source holds a stored reference to the target
dashed line, open arrow	dependency	transient use only (parameter, return, local); no stored field

6.2 Diagram 1: structural spine

The request path from the web workspace through the chat coordinator and LangGraph workflow to the specialists, tools and stores.

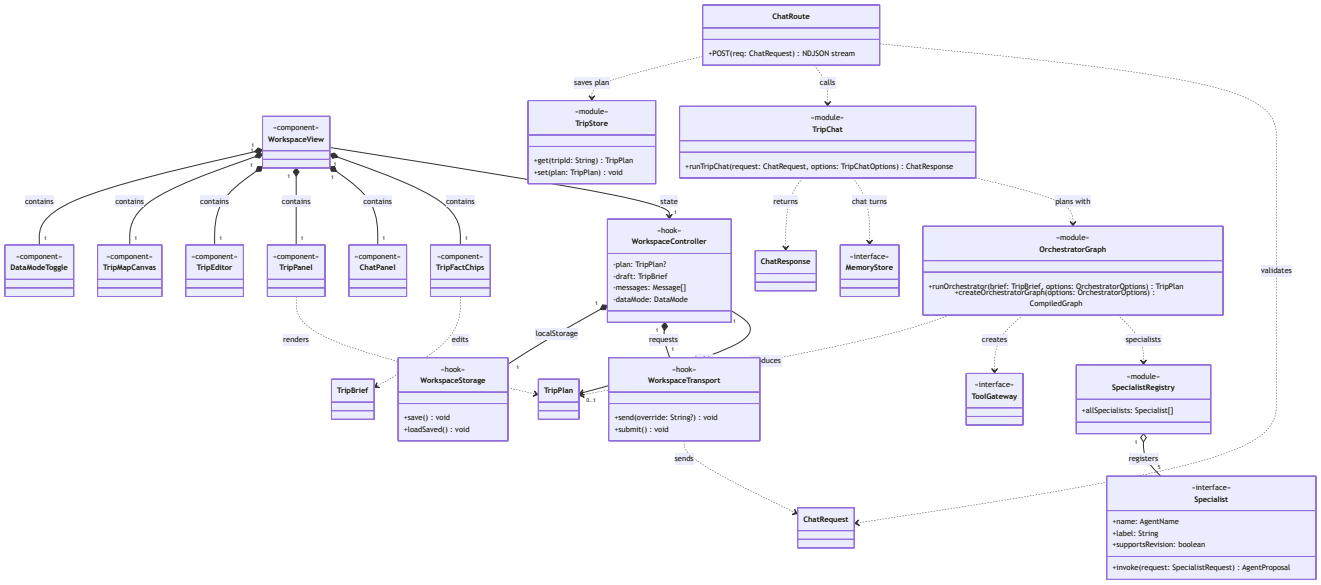


Figure 6.1. Diagram 1: structural spine.

6.3 Diagram 2: domain model

The shared contracts: `TripBrief`, `AgentProposal`, `ProposalItem`, `RevisionRequest`, `TripPlan` and `TripSection`.

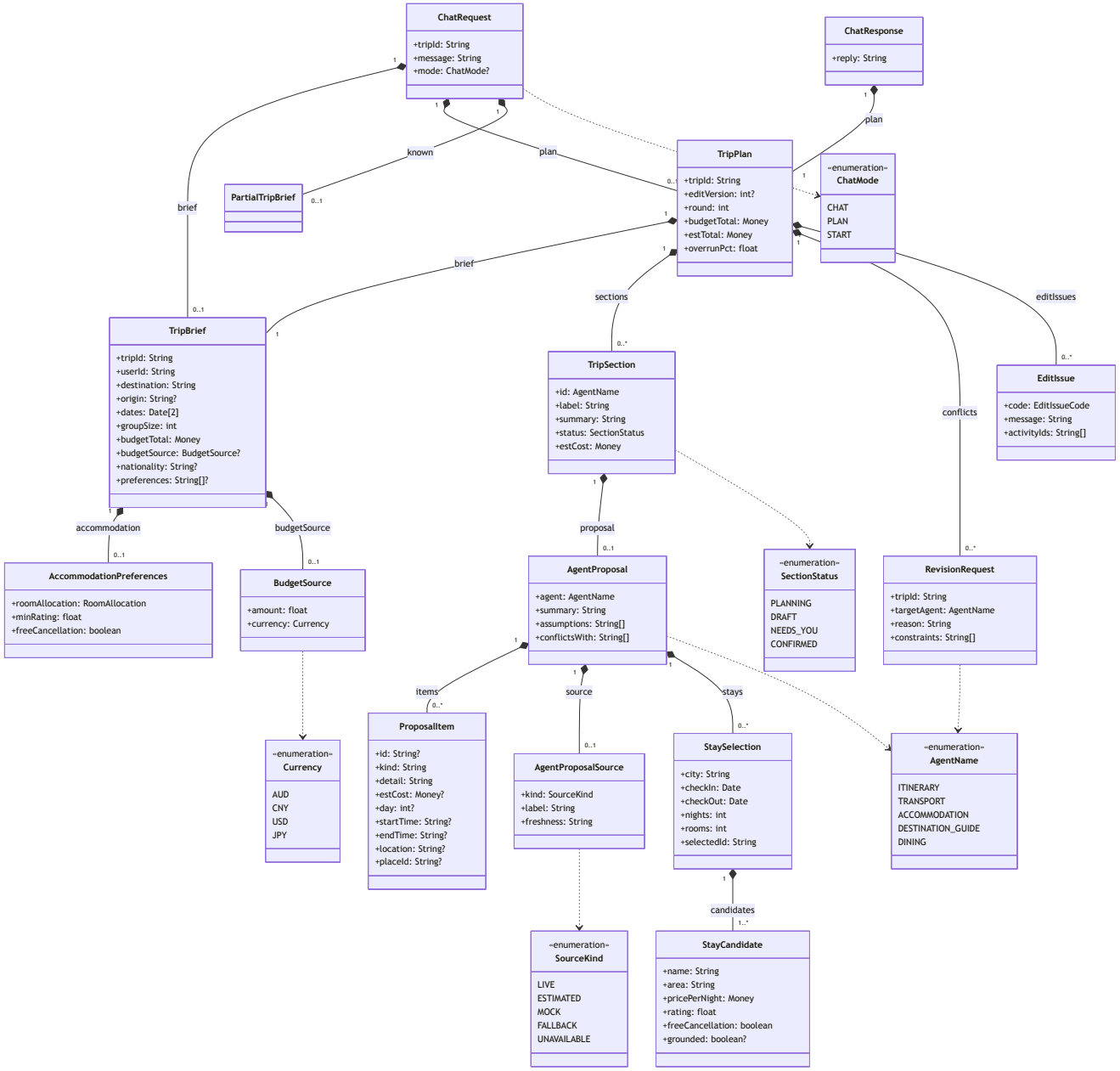


Figure 6.2. Diagram 2: domain model.

6.4 Diagram 3: specialists and orchestration

The `Specialist` interface, its five realisations, the registry and the orchestrator graph.

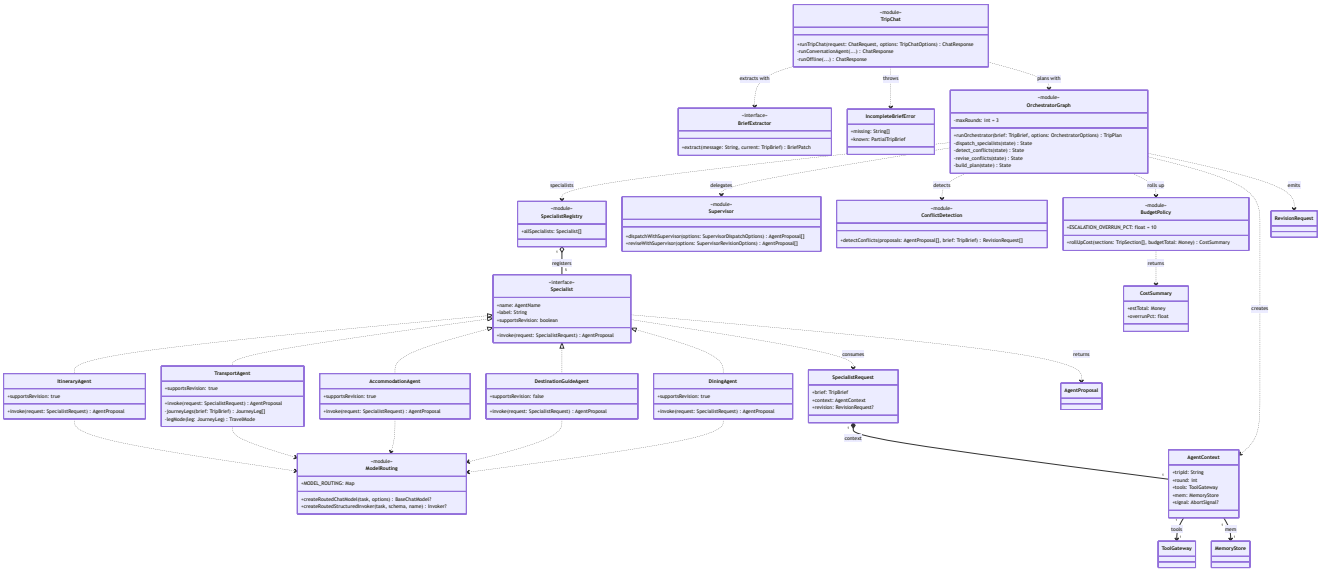


Figure 6.3. Diagram 3: specialists and orchestration.

6.5 Diagram 4: ports, adapters and infrastructure

Hexagonal ports in `packages/shared` and the adapters that realise them.

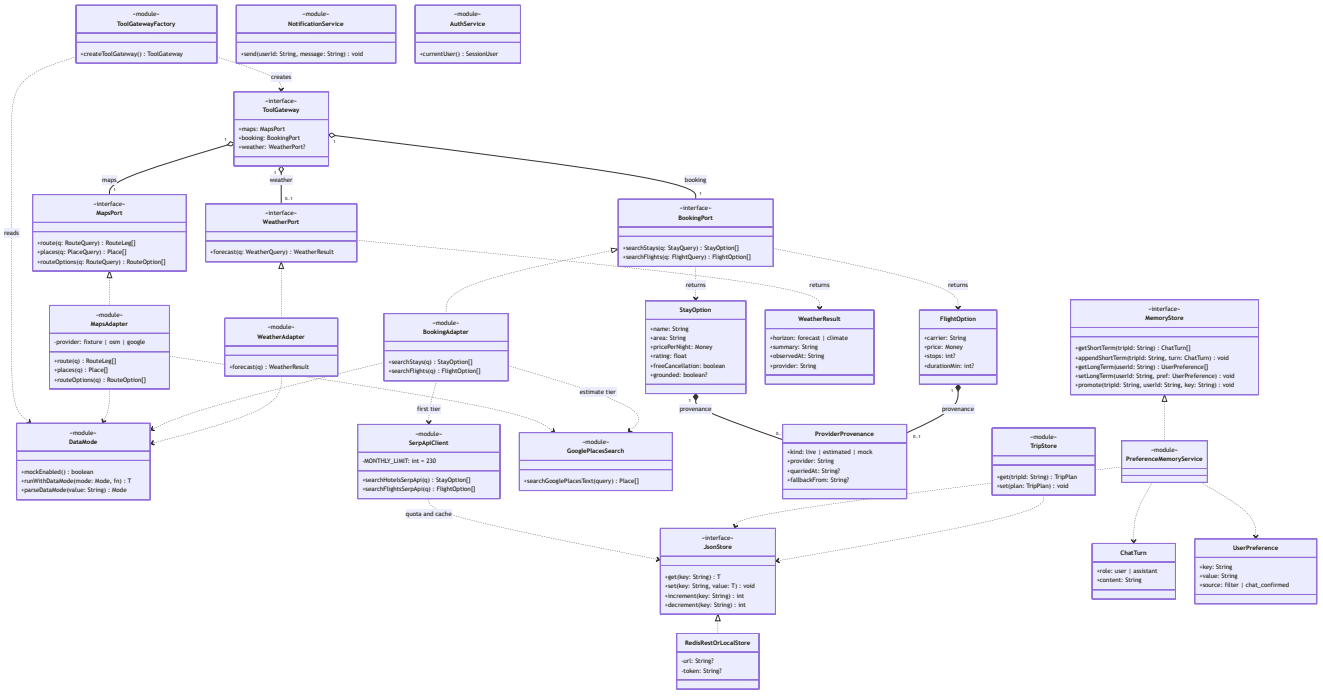


Figure 6.4. Diagram 4: ports, adapters and infrastructure.

6.6 Diagram 5: use cases traced onto the model

Which classes realise each use case.

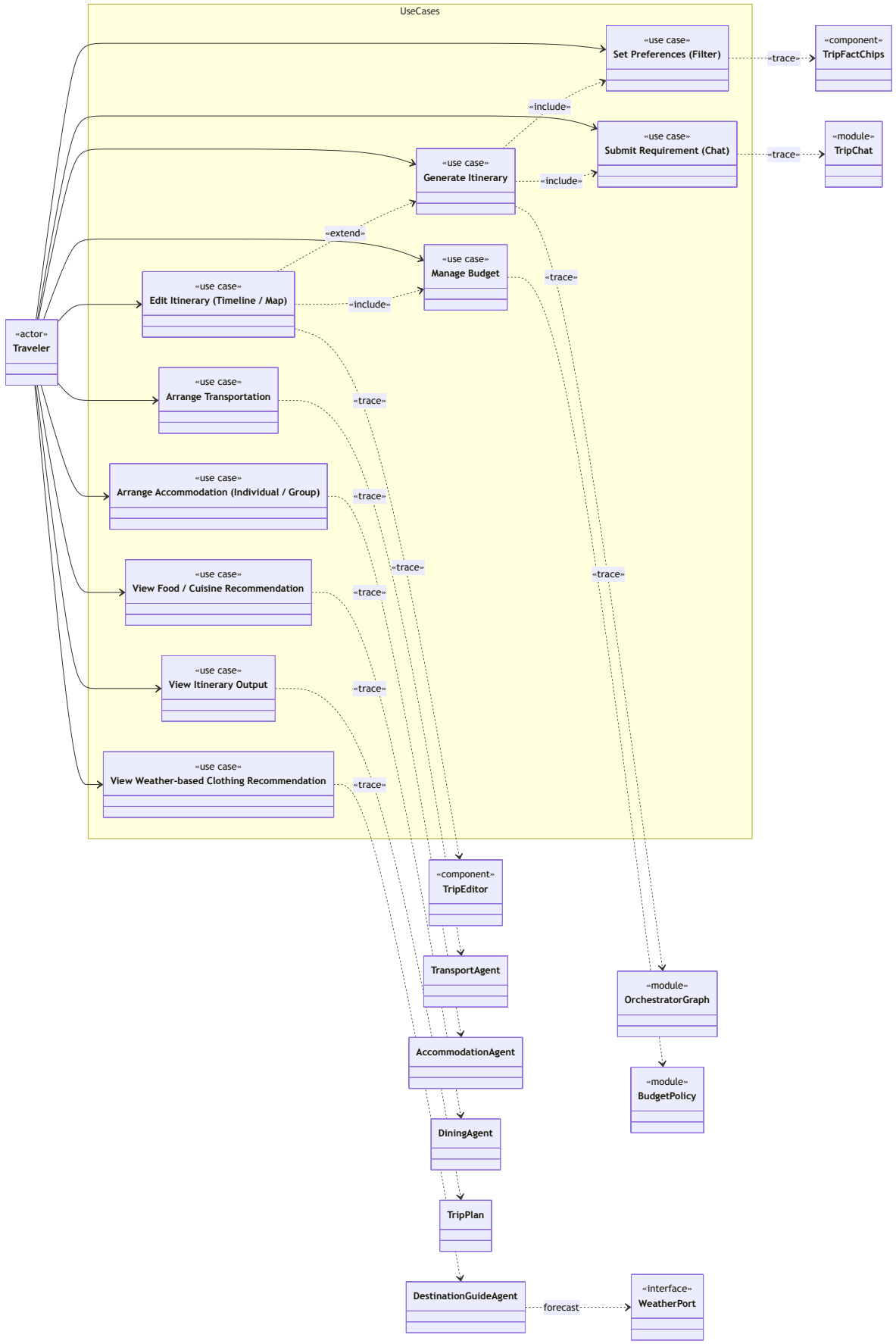
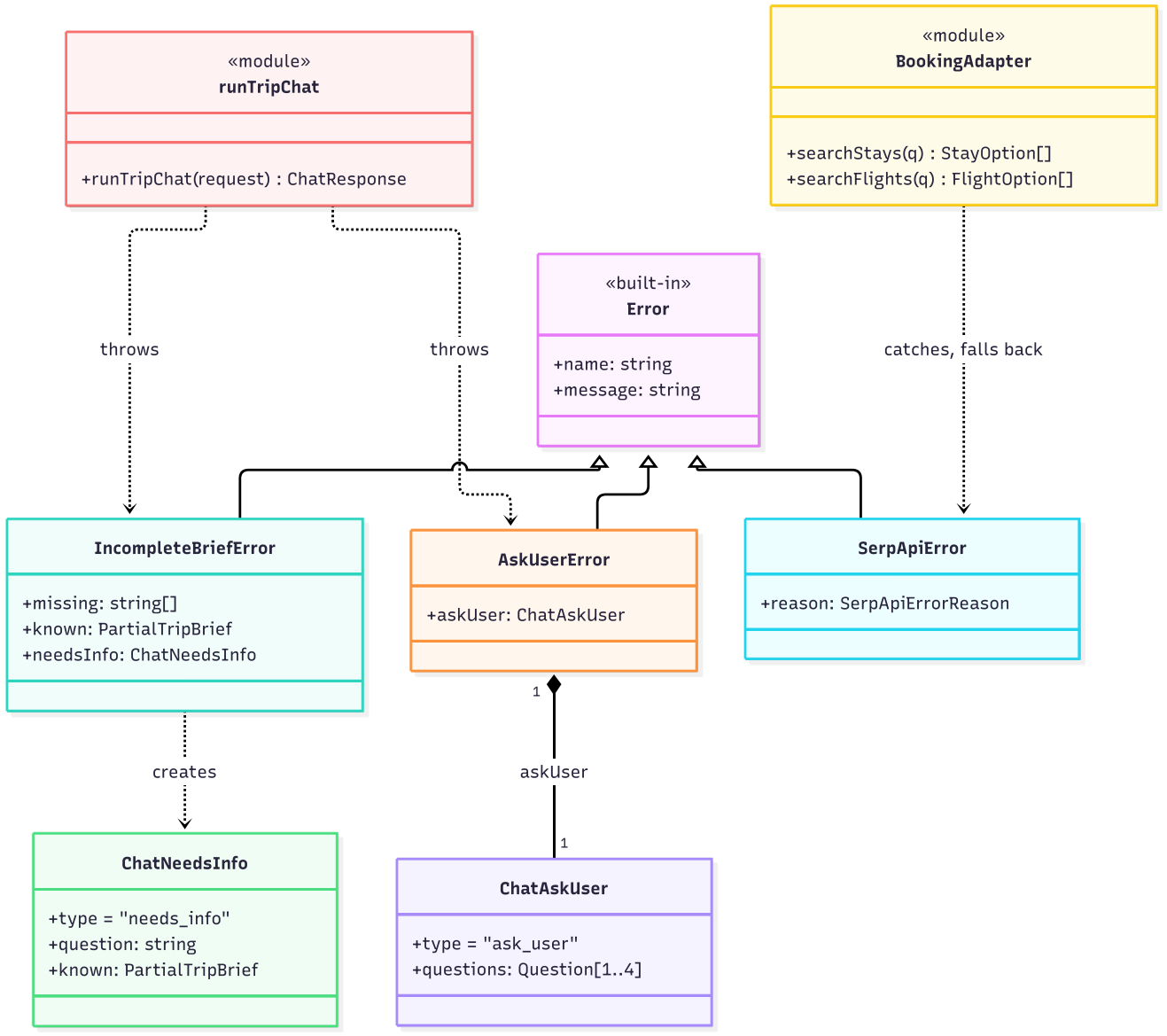


Figure 6.5. Diagram 5: use cases traced onto the model.

6.7 Diagram 6: generalisation

The five class diagrams in above already show composition, aggregation, multiplicity, interfaces and realisation (`Specialist <|.. ItineraryAgent`, `MapsPort <|.. MapsAdapter`, …). The criteria also name **generalisation**, which the existing diagrams do not draw. The code does have real inheritance: the typed errors that turn a failed or unfinished turn into a specific response frame. This is **Diagram 6** of the class model.



Relationship	Kind	Rationale
`Error <|-- IncompleteBriefError`	generalisation	"Not enough to plan yet" is a non-plan outcome, not a crash; the API route catches it by type and streams a `needs_info` frame with the fields understood so far.
`Error <|-- AskUserError`	generalisation	A clarifying question with 1–4 concrete choices, streamed as `ask_user`.
`Error <|-- SerpApiError`	generalisation	A provider failure with a typed `reason` (quota, no results, …) so the booking adapter can fall back to Google Places estimates and label the provenance.

Design note. Subclassing `Error` lets `runTripChat` end a turn early from deep inside the LangGraph or LangChain call stack, and lets the route handler distinguish outcomes with `instanceof` instead of string matching.

6.8 Key associations and multiplicity

Read *source* → *target*.

Source	Target	Type	Mult.	Meaning
`WorkspaceView`	panels, editor, map, data-mode toggle	composition	1 → 1	the workspace renders one of each view
`WorkspaceController`	`TripPlan`	association	1 → 0..1	holds the current plan; none before the first plan
`WorkspaceController`	`WorkspaceTransport` / `WorkspaceStorage`	composition	1 → 1	request and `localStorage` hooks used only by this controller
`ChatRoute`	`TripChat`, `TripStore`	dependency	—	handles one request, then saves the resulting plan
`ChatResponse`	`TripPlan`	composition	1 → 1	a response carries a full plan
`ChatRequest`	`TripBrief` / `TripPlan` / `PartialTripBrief`	composition	1 → 0..1	the browser sends its current state with each message
`OrchestratorGraph`	`Supervisor`, `ConflictDetection`, `BudgetPolicy`	dependency	—	functions called from graph nodes
`SpecialistRegistry`	`Specialist`	aggregation	1 → 5	references module-level agents; no lifecycle ownership
`SpecialistRequest`	`AgentContext`	composition	1 → 1	each call carries its own context
`AgentContext`	`ToolGateway` / `MemoryStore`	association	1 → 1	injected — the specialist's only route to tools and memory
`TripPlan`	`TripBrief`	composition	1 → 1	embeds the brief it answers
`TripPlan`	`TripSection`	composition	1 → 0..*	one section per specialist that returned a proposal
`TripPlan`	`RevisionRequest`	composition	1 → 0..*	conflicts still unresolved after the last round
`TripSection`	`AgentProposal`	composition	1 → 0..1	the proposal the section was built from, for drill-down
`AgentProposal`	`ProposalItem`	composition	1 → 0..*	a proposal is its list of line items
`AgentProposal`	`AgentProposalSource`	composition	1 → 0..1	where the data came from: live, estimated, mock, fallback, unavailable

Source	Target	Type	Mult.	Meaning
`StaySelection`	`StayCandidate`	composition	1 → 1..*	the chosen stay and its alternatives
`ToolGateway`	`MapsPort` / `BookingPort` / `WeatherPort`	aggregation	1 → 1, 1, 0..1	one port of each kind; weather is optional
`StayOption` / `FlightOption`	`ProviderProvenance`	composition	1 → 0..1	which provider answered and whether it was a fallback
`PreferenceMemoryService` / `TripStore`	`JsonStore`	dependency	—	every read and write goes through the store

6.9 Interfaces and realisation

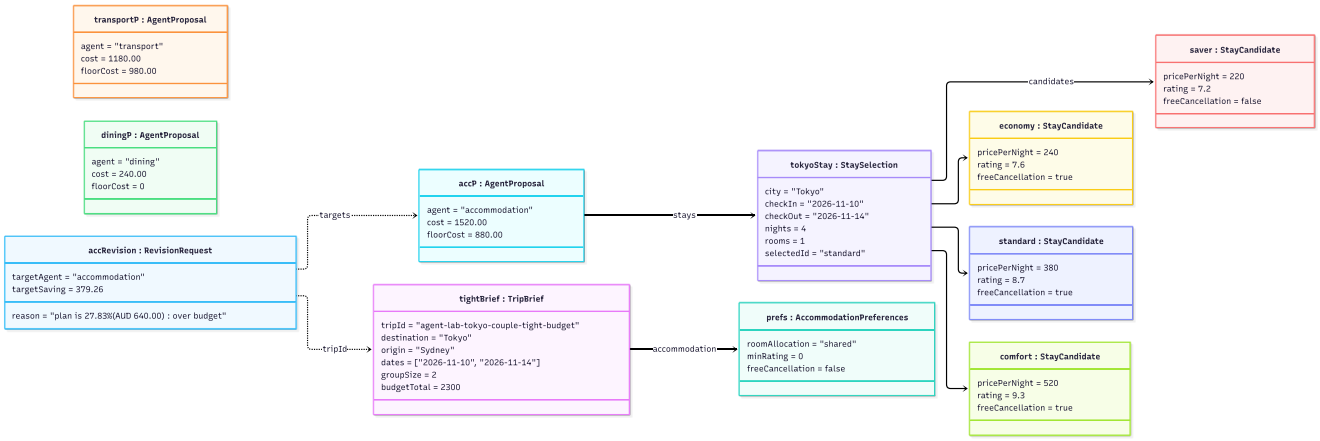
Interface	Realised by	Note
`Specialist`	`ItineraryAgent`, `TransportAgent`, `AccommodationAgent`, `DestinationGuideAgent`, `DiningAgent`	one `invoke(SpecialistRequest)`; only the destination guide cannot revise
`ToolGateway`	object returned by `createToolGateway()`	one per planning run
`MapsPort`	`MapsAdapter` (`maps.ts`)	owner B; fixtures, OpenStreetMap or Google
`BookingPort`	`BookingAdapter` (`booking.ts`)	owner C; SerpApi, then Google Places estimate, or fixtures; no payment
`WeatherPort`	`WeatherAdapter` (`weather.ts`)	owner D; Google Weather, Open-Meteo or climate context
`MemoryStore`	`PreferenceMemoryService` (`memory`)	owner E
`JsonStore`	`RedisRestOrLocalStore` (`createJsonStore()`)	owner E; Redis REST when configured, process memory otherwise
`BriefExtractor`	model extractor in `chat.ts`, or `extractBriefPatchLocally` without a key	owner A

`NotificationService` and `AuthService` are stubs: notifications are logged, and every request is the demo user.

7. Complex structure: object, collaboration and structured class

7.1 Object diagram

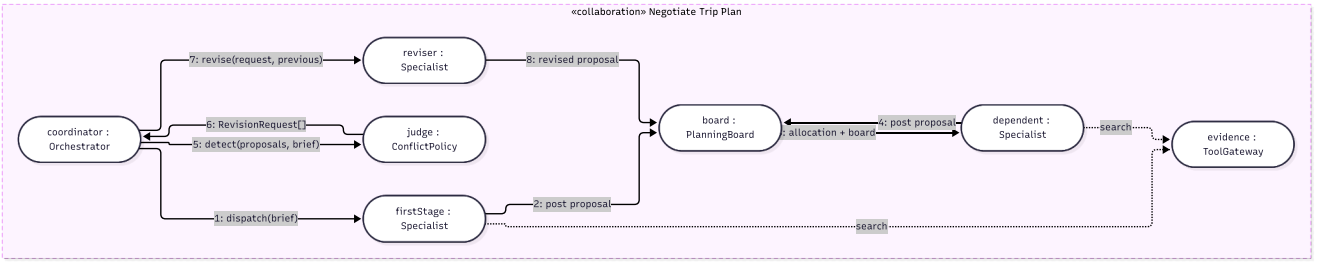
A snapshot of the planning board at the end of round 1 of `detect_conflicts`, for the Agent Lab's `tokyo-couple-tight-budget` brief. The brief and the four stay candidates are the real fixture values (`agent-lab/scenarios.ts`, `tools/src/booking.ts`). **Transport and dining costs are illustrative**: re-run the lab and replace them if you want exact figures.



How to read it: the couple shares one room (⌈2 / 2⌉ = 1). The first choice is *standard*, the cheapest candidate rated ≥ 8 with free cancellation, at 1 × 4 × 380 = AUD 1,520. The floor is *saver*, at 4 × 220 = AUD 880. The total of AUD 2,940 is AUD 640 (27.83%) over the AUD 2,300 budget. The overrun is spread over each section's room to cut (transport 200, accommodation 640, dining 240, 1,080 in total), so accommodation is asked to save 640 × 640 / 1,080 = AUD 379.26. In round 2 its allocation is AUD 1,140.74. A budget revision takes the cheapest eligible candidate, which is *saver* (AUD 880): free cancellation was not a confirmed preference, so *saver* is still eligible. Had the traveller required free cancellation, *saver* would be filtered out and *economy* (AUD 960) chosen instead.

7.2 Collaboration: «collaboration» Negotiate Trip Plan

Roles, not classes: any `Specialist` can play a role, and the connectors are the links the roles use during one planning turn.

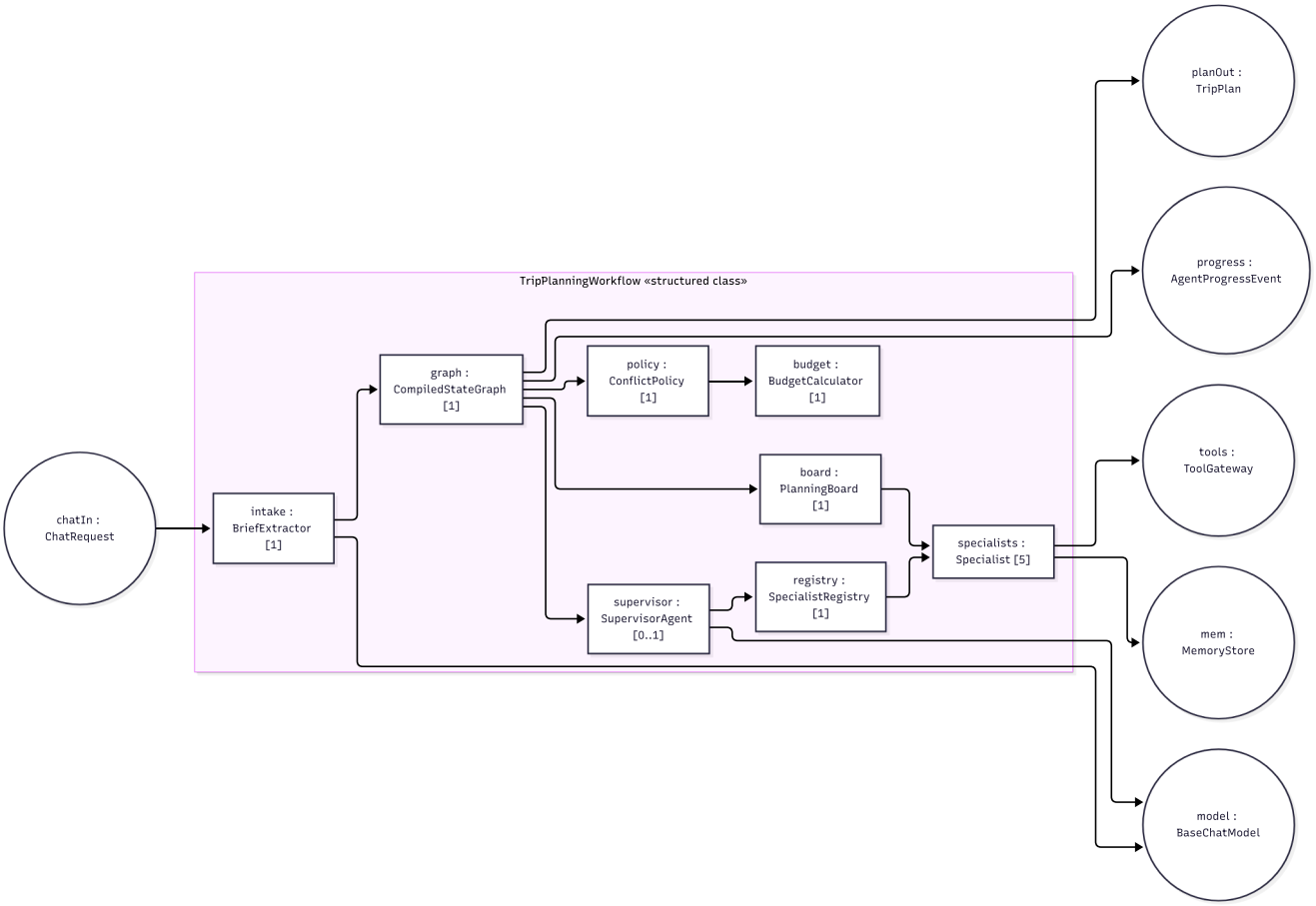


Role	Played by (in `main`)	Responsibility in the collaboration
coordinator	LangGraph workflow plus supervisor agent	Decides when to dispatch, detect, revise and stop.

Role	Played by (in `main`)	Responsibility in the collaboration
board	`board.ts`	Orders the stages (transport → accommodation → itinerary and dining) and computes each allocation.
firstStage	Transport (also Destination guide, which has no budget share)	Plans without waiting for anyone.
dependent	Accommodation, Itinerary, Dining	Waits for earlier stages and plans within its allocation.
judge	`detectConflicts` and `rollUpCost` (member C)	Turns proposals into targeted `RevisionRequest`s.
reviser	Any specialist with `supportsRevision`	Revises only its own section, from its previous proposal.
evidence	`ToolGateway` (maps, booking, weather)	Supplies grounded places, prices and forecasts.

7.3 Structured class: TripPlanningWorkflow

The internal structure of the orchestrator as a composite: its parts, the ports it exposes and requires, and the connectors between parts.



Element	Kind	Notes
`chatIn`, `planOut`, `progress`	provided ports	`POST /api/chat` request in; NDJSON progress frames and a final `{ reply, plan }` out.
`tools`, `mem`, `model`	required ports	Injected through `OrchestratorOptions` and `AgentContext`; tests pass fakes.
`supervisor [0..1]`	part	Absent when no model is configured; the graph then dispatches deterministically.
`specialists [5]`	part	Itinerary, Transport, Accommodation, Destination guide, Dining.

Element	Kind	Notes
`policy`, `budget`	parts	Member C's `detectConflicts`, `rollUpCost` and `minimumCost`.

8. Behaviour models (individual)

Each member's activity, sequence and state machine diagrams come from their own ad hoc requirement and use case specification, and show where the LLM sits and which deterministic code checks it.

8.1 Member A (`@Lilstanie`, coordination and orchestration)

All three diagrams are built from ad hoc requirement **AH-A1** and use case **UC-A1 Generate Itinerary** (§2.2, §4.2). Each one marks where the LLM acts and which code validates what it returns.

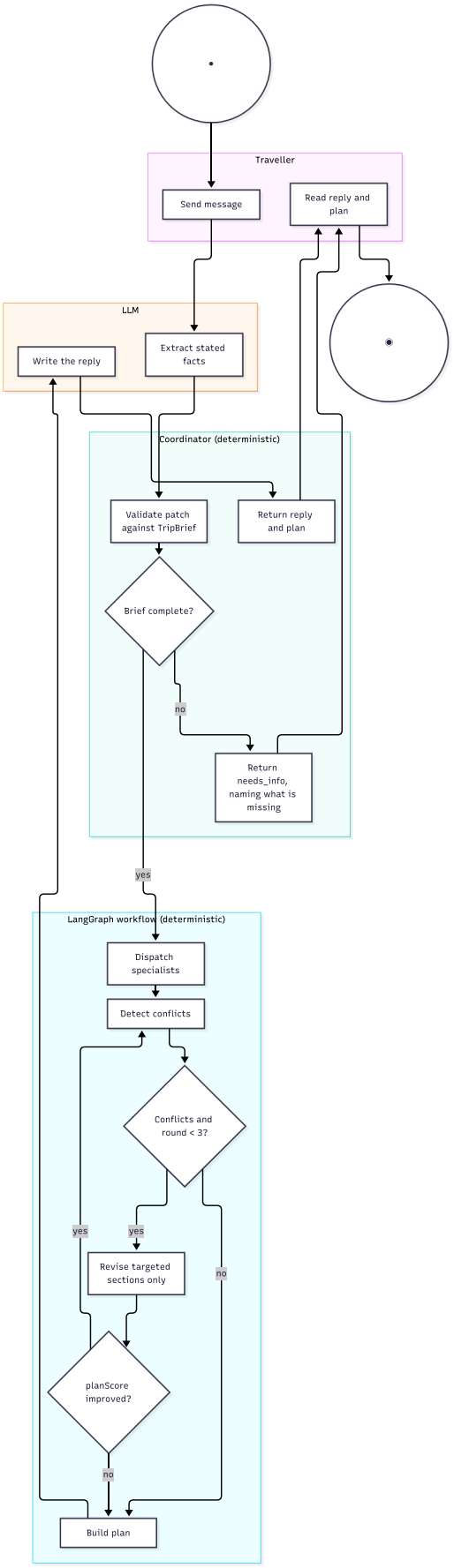
One planning turn makes three LLM calls. Each has a deterministic path that takes over when the model is unavailable or returns invalid output:

LLM call	What it decides	Deterministic guard
Brief extraction (`update_trip_brief`)	Which trip facts the message states	`BriefPatchSchema` → `applyBriefPatch` → `TripBrief` Zod parse. Facts the traveller did not state are reported as missing, not inferred
Supervisor delegation	Which specialists to call, with what objective	Fixed specialist registry. The itinerary specialist is always delegated, and a failed supervisor falls back to dispatching all five
Reply writing	The prose the traveller reads	With no model configured, `fallbackReplyFor(plan)` writes it from the plan

The plan is not generated by the model. `build_plan` assembles it from the validated proposals.

8.1.1 Activity diagram: Generate Itinerary (UC-A1)

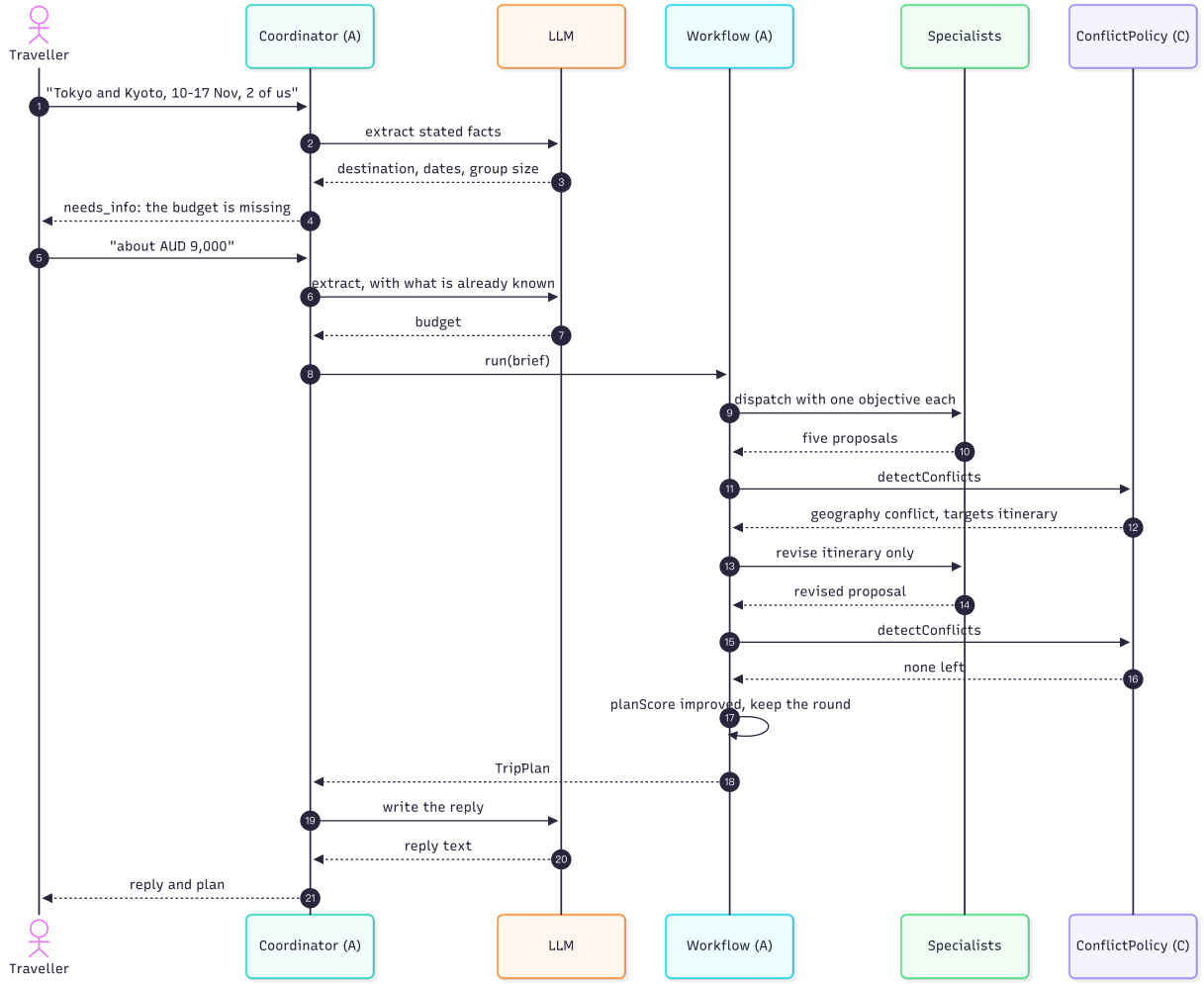
The swimlanes are the five participants. Rounds 2 and 3 re-enter *Detect conflicts*.



Without a model key, only the three LLM boxes drop out. Extraction uses the facts already stated, delegation dispatches all five specialists, and `fallbackReplyFor` writes the reply.

8.1.2 Sequence diagram: one turn with a clarifying question, then a revised plan (UC-A1, extensions 3a and 7a)

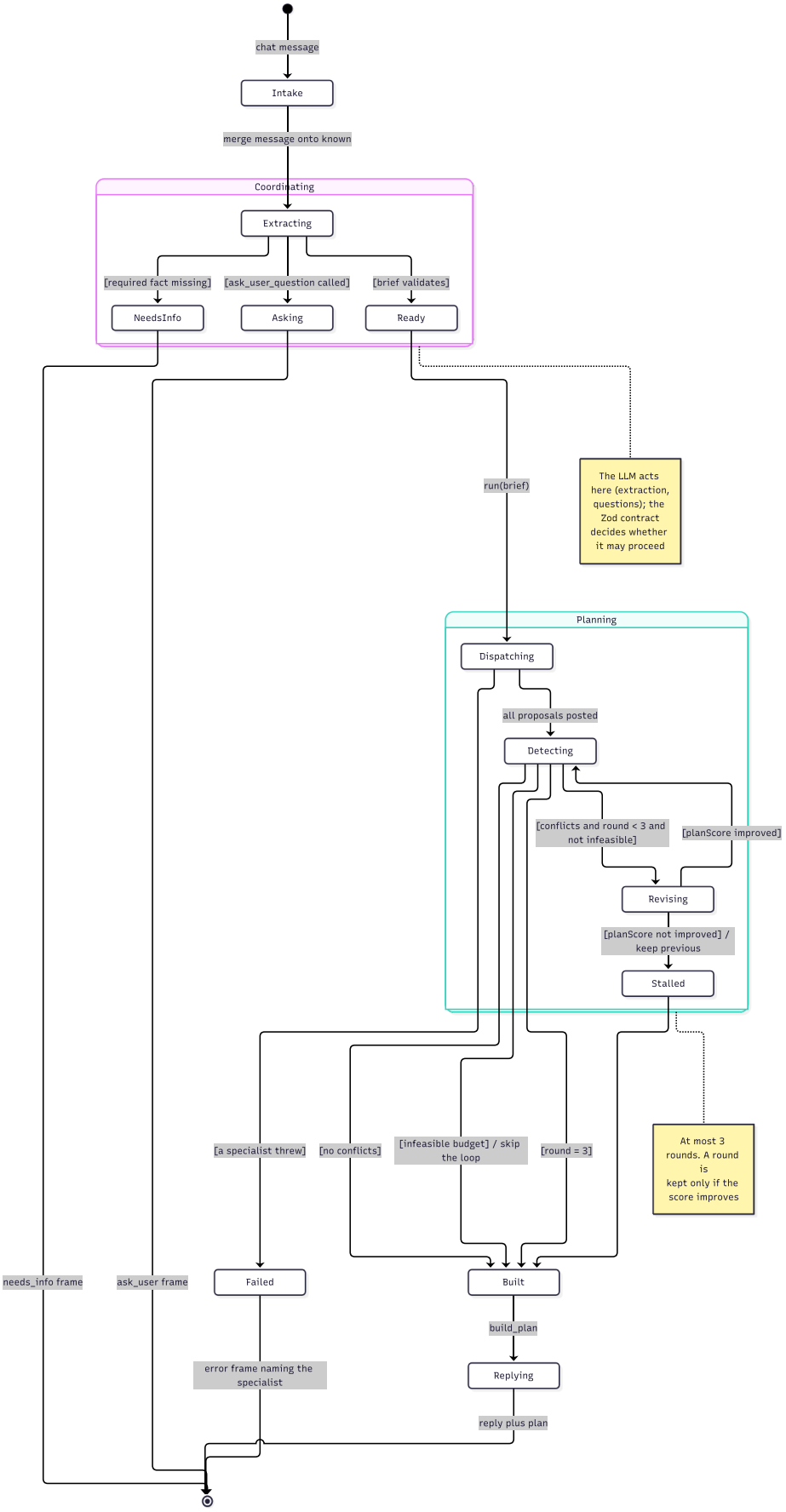
The first message leaves out the budget, so the coordinator asks for it before planning starts. The first round returns a geography conflict, one targeted revision fixes it, and the plan score improves.



If the score had not improved, `revise_conflicts` sets `stalled` and the previous proposals are kept. If the conflict had been `infeasible budget`, `routeAfterDetection` skips the loop.

8.1.3 State machine: one planning turn (UC-A1)

This is the turn as the orchestrator sees it. Member C's state machine covers a single *section* inside `Detecting`. This one covers the whole turn.



State	What the traveller sees	Entry / exit behaviour
Intake, Extracting	"thinking"	The message is merged onto `known`, so a follow-up does not repeat earlier facts.

State	What the traveller sees	Entry / exit behaviour
NeedsInfo	The question in chat	No plan exists yet. `IncompleteBriefError` names the missing fields, and required facts are not given default values.
Asking	2–4 clickable options	At most one question per turn, and no other tool runs after it.
Dispatching	A progress row per subagent	The supervisor chooses which specialists run. The itinerary specialist always runs.
Detecting	"checking budget and schedules"	Member C's `detectConflicts` returns the revision requests.
Revising	"revising affected sections"	Only the targeted agents re-run, each given its previous proposal.
Stalled	—	The round is discarded and the earlier proposals are kept.
Built	Sections `draft` or `needs_you`	The plan is assembled from the validated proposals.
Failed	Error naming the specialist	No partial plan is returned.

8.1.4 How this behaviour is measured

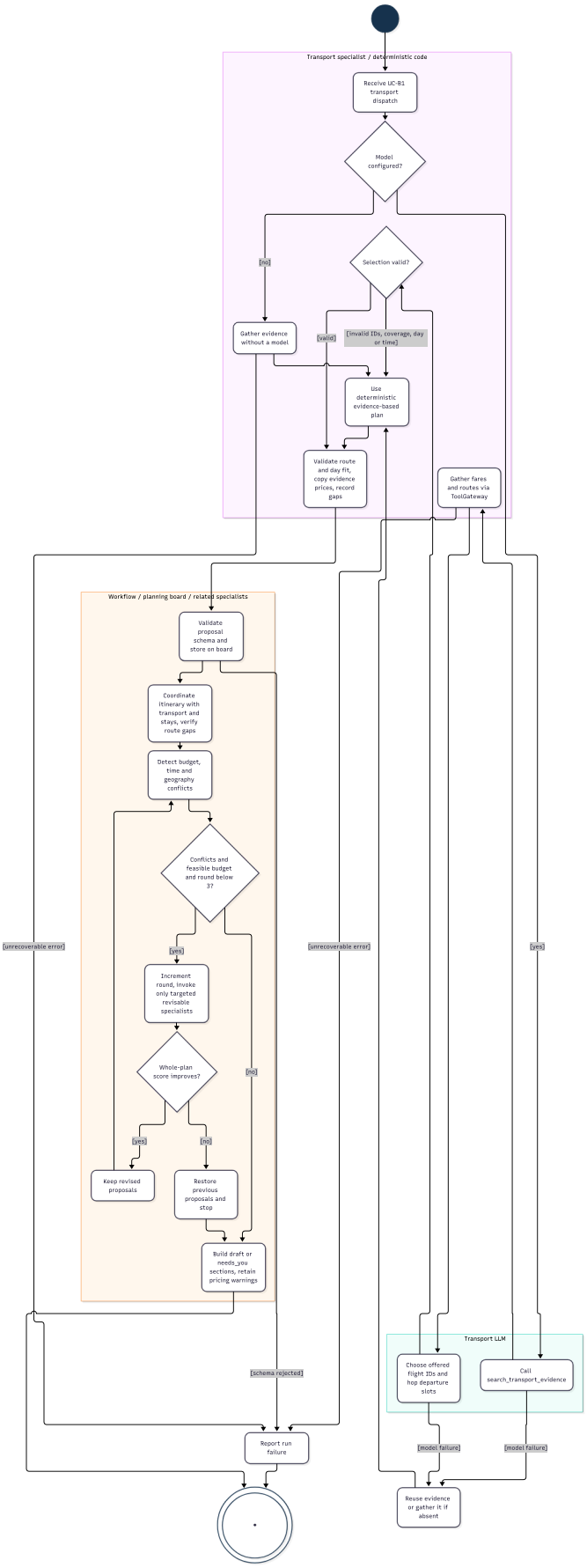
`packages/orchestrator/src/agent-lab/` replays fixed scenarios against the orchestrator with injected faults: a specialist timing out, a provider failing, a model returning output that does not match the schema. Each run records the number of rounds, the conflicts raised, token usage and an evaluator score (R-A12). This is how the transitions above are tested under failure conditions.

8.2 Member B (`@fonever2`, itinerary and transport)

These models derive from **AH-B1 / R-B1–R-B6** (§2.3.2) and **UC-B1 Arrange Transportation** (§4.2). They describe the current team implementation in B's assigned domain, not exclusive code authorship. The specialist boundary is `Specialist.invoke`.

8.2.1 Activity diagram

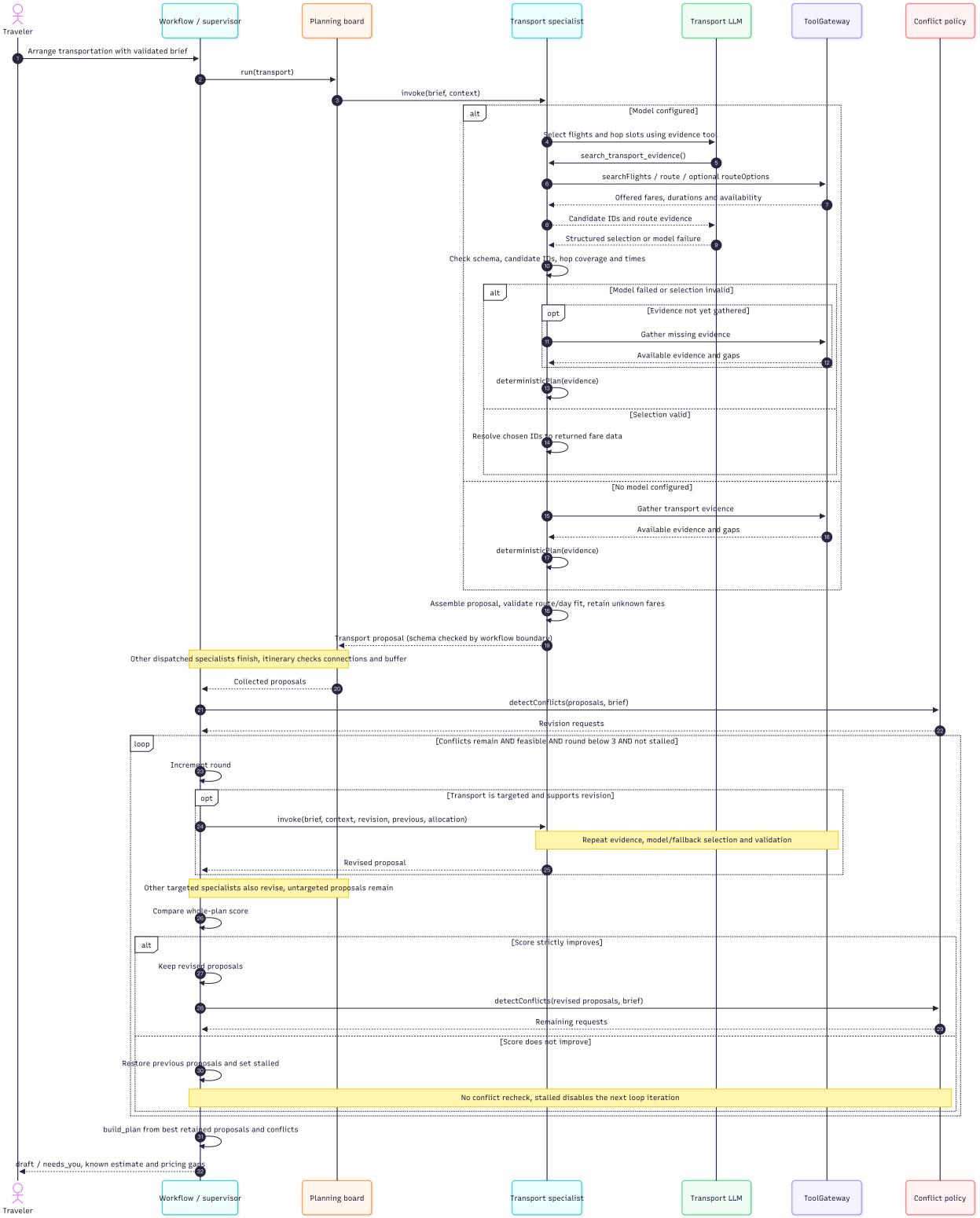
Rounded actions, guarded decisions, a filled initial node and a double-ring final node express the activity model in Mermaid. Responsibility regions separate the Transport LLM from deterministic code and the workflow. The revision action expands the targeted specialist's planning operation; it does not imply rerunning every agent. Cancellation can terminate any executing action and is omitted for readability.



8.2.2 Sequence diagram

The model-call branch shows rejected output falling back after validation. A separate no-model branch gathers evidence directly. The nested optional fragment invokes Transport only when targeted. A non-

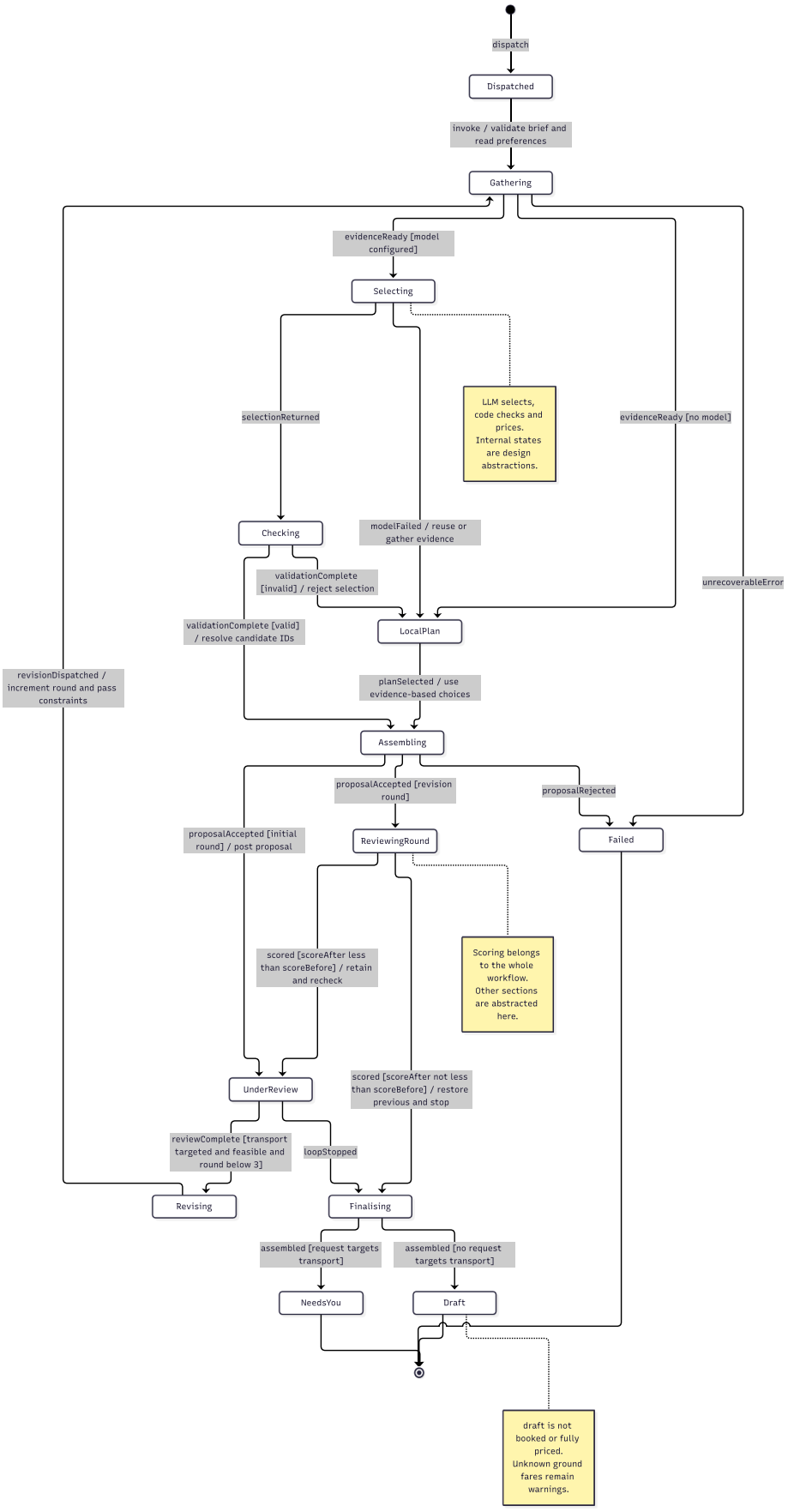
improving revision restores the previous proposals and exits the revision loop before final assembly; it does not recheck rejected proposals. A model failure before its search tool is called skips that tool exchange and gathers evidence in the fallback. Provider and cancellation errors that end the run are specified in UC-B1 rather than expanded here.



8.2.3 State machine

The subject is the transport proposal within one planning run. Transition labels follow event [guard] / action. Internal states are conceptual, not persisted enums; only `draft` and `needs_you` map to final

section status. Whole-plan score comparison and round limits belong to the workflow. While other sections revise, an untargeted transport proposal remains under review until the workflow stops. An abort can terminate any non-final state; those repeated transitions are omitted.



8.2.4 Implementation boundaries

- The LLM chooses offered flight IDs and hop departure slots. Deterministic code checks candidate membership, one fare per priced hop, required hop coverage, day bounds, time format and route/day fit before assembling a proposal.
- Unknown ground fares omit `estCost` and remain warnings; a required flight without a valid fare still records a conflict. Traveller-arranged flights are exempt. An unavailable chosen ground mode is explicitly disclosed rather than silently substituted.
- Itinerary planning checks connections against provider durations plus a 15-minute arrival buffer. Transport durations are gathered before model scheduling; a model-changed departure time does not trigger another time-sensitive route search.
- The default limit is three total rounds, including the first. Only targeted revisable specialists rerun. Non-improving rounds are discarded. There is no purchase, booking or approval checkpoint in this use case.
- Generated figures are for full-size report viewing; use the SVGs for zooming. Their detailed labels are not intended to be read as three tiny slide thumbnails.

8.2.5 AI acknowledgement

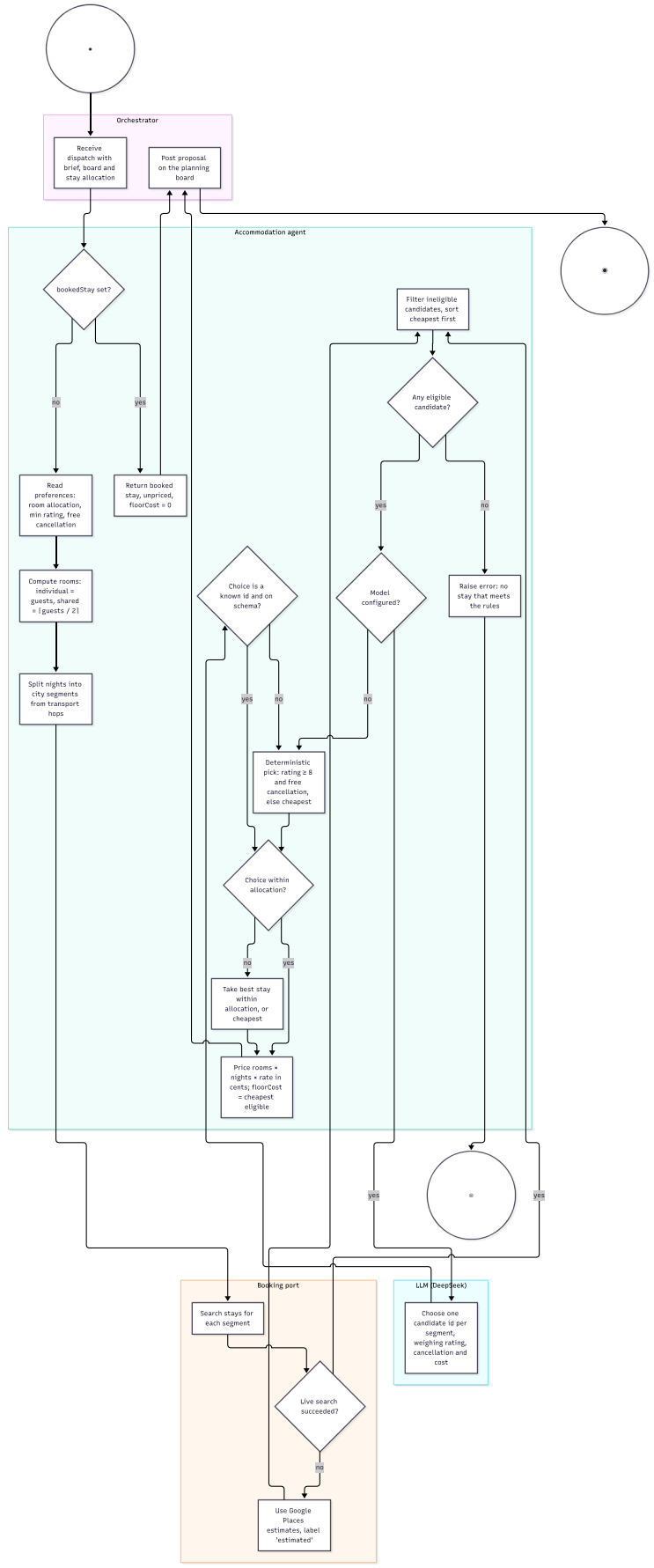
OpenAI Codex assisted with source inspection, the requirement and use-case draft, and diagram preparation. Tingsong Jin must review the individual submission and be able to explain the models. This statement does not assert that all current team code was implemented by B.

8.3 Member C (`@HeadmasterEggy`, accommodation and budget)

All three diagrams come from ad hoc requirement **AH-C1** and the use case specifications **UC-C1 Arrange Accommodation** and **UC-C2 Manage Budget** (§2.2, §4.2). Each focuses on where the LLM sits and what deterministic code guards it, as the brief asks.

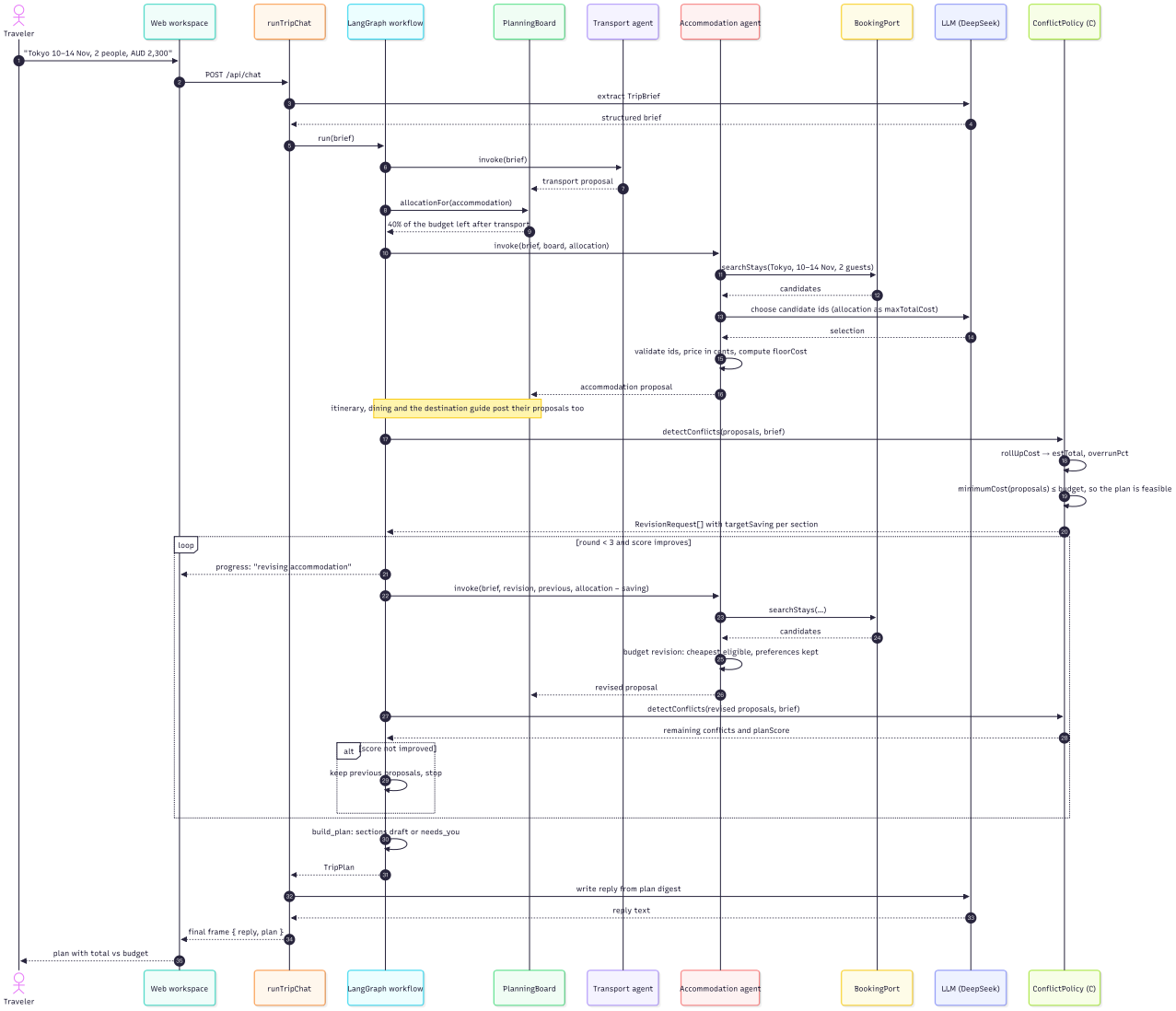
8.3.1 Activity diagram: Arrange Accommodation (UC-C1)

Swimlanes are the four participants. The LLM is used once, in *Choose one candidate id per segment*, and its output is checked before anything is priced.



8.3.2 Sequence diagram: budget overrun and targeted revision (UC-C2, extension 2a)

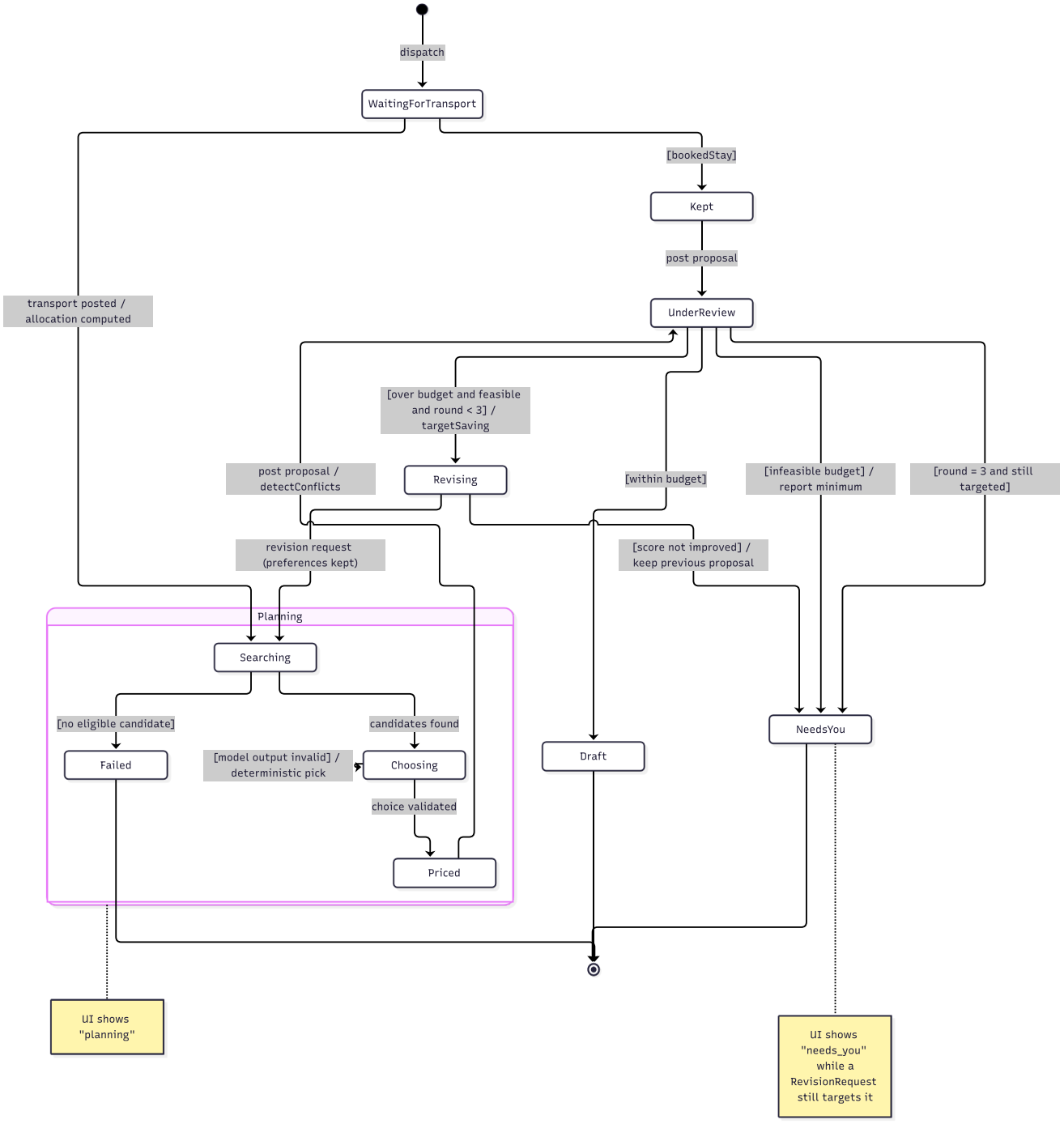
One planning turn in which the first round goes over budget and the accommodation agent is asked to cut its cost.



If `minimumCost` is above the budget (extension 2b), `detectConflicts` instead returns one `infeasible budget` conflict, the loop is skipped, and the reply names the minimum budget needed.

8.3.3 State machine: accommodation section within a planning turn (UC-C1 and UC-C2)

The section's visible status (`planning`, `draft`, `needs_you`) is derived from these internal states.



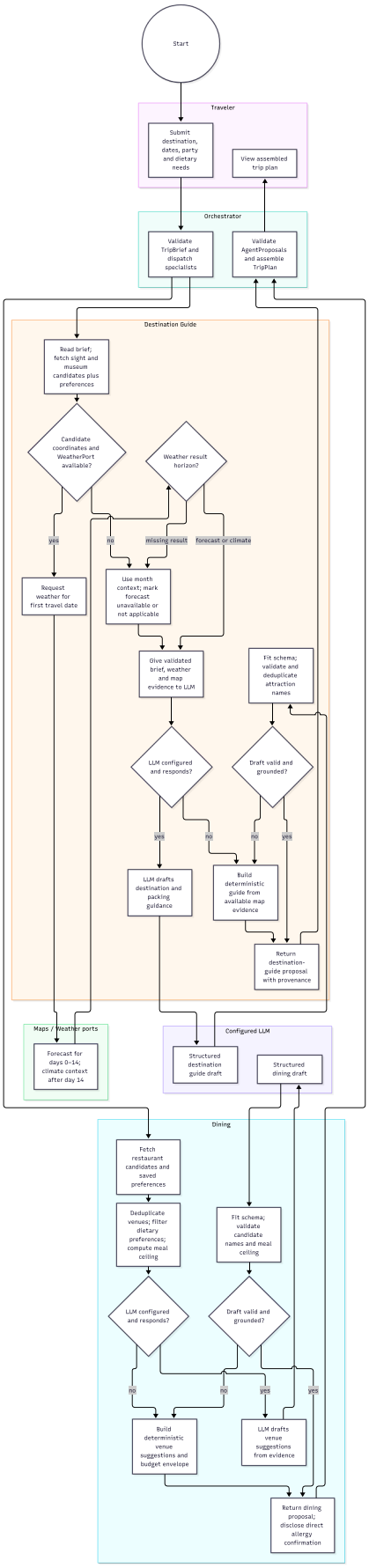
State	UI status	Entry / exit behaviour
WaitingForTransport	planning	Waits only if transport was started in the same turn.
Searching, Choosing, Priced	planning	The LLM acts only in Choosing; invalid output loops back through the deterministic pick.
Kept	planning	Booked stay, no search; floorCost 0 so it is never asked to cut.
UnderReview	planning	Member C's `detectConflicts` decides the next state.
Revising	planning	Same preferences, new allocation; the round is kept only if `planScore` improves.
Draft	draft	No revision request targets the section.
NeedsYou	needs_you	The traveller decides: raise the budget, change dates or edit the plan.
Failed	—	The run stops and names accommodation as the failing specialist.

8.4 Member D

All three diagrams come from **AH-D1** and **UC-D1 View Weather-based Clothing Recommendation**. The destination-guide and dining specialists run as part of one trip plan. The LLM drafts from tool evidence; deterministic code validates schema, candidate names and meal budget before proposals reach the plan. Weather provenance distinguishes a forecast from historical climate context.

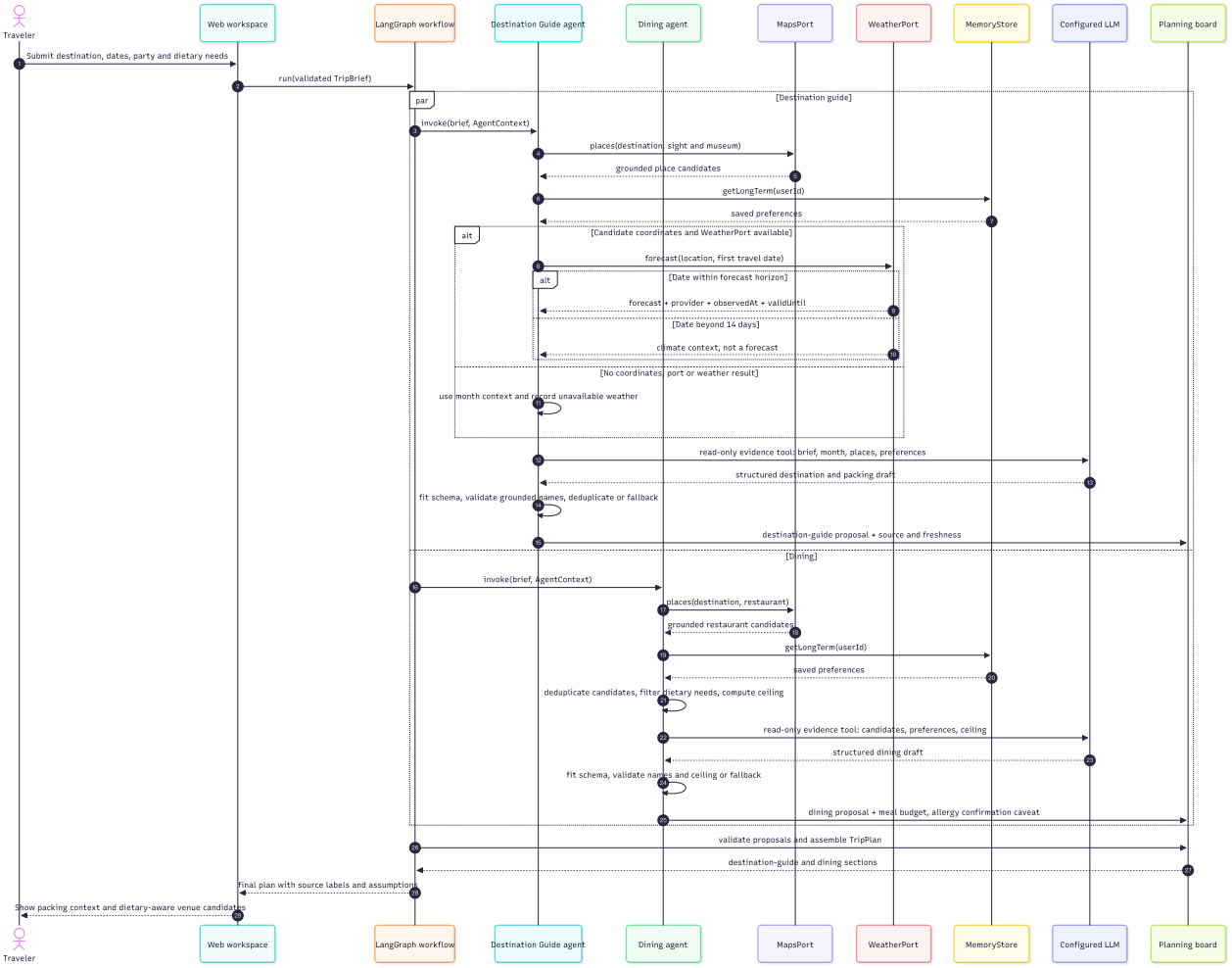
8.4.1 Activity diagram: weather and dietary guidance (UC-D1)

The specialist paths run independently. Weather failure degrades only the guide's weather detail; invalid model output uses deterministic, evidence-bounded guidance.



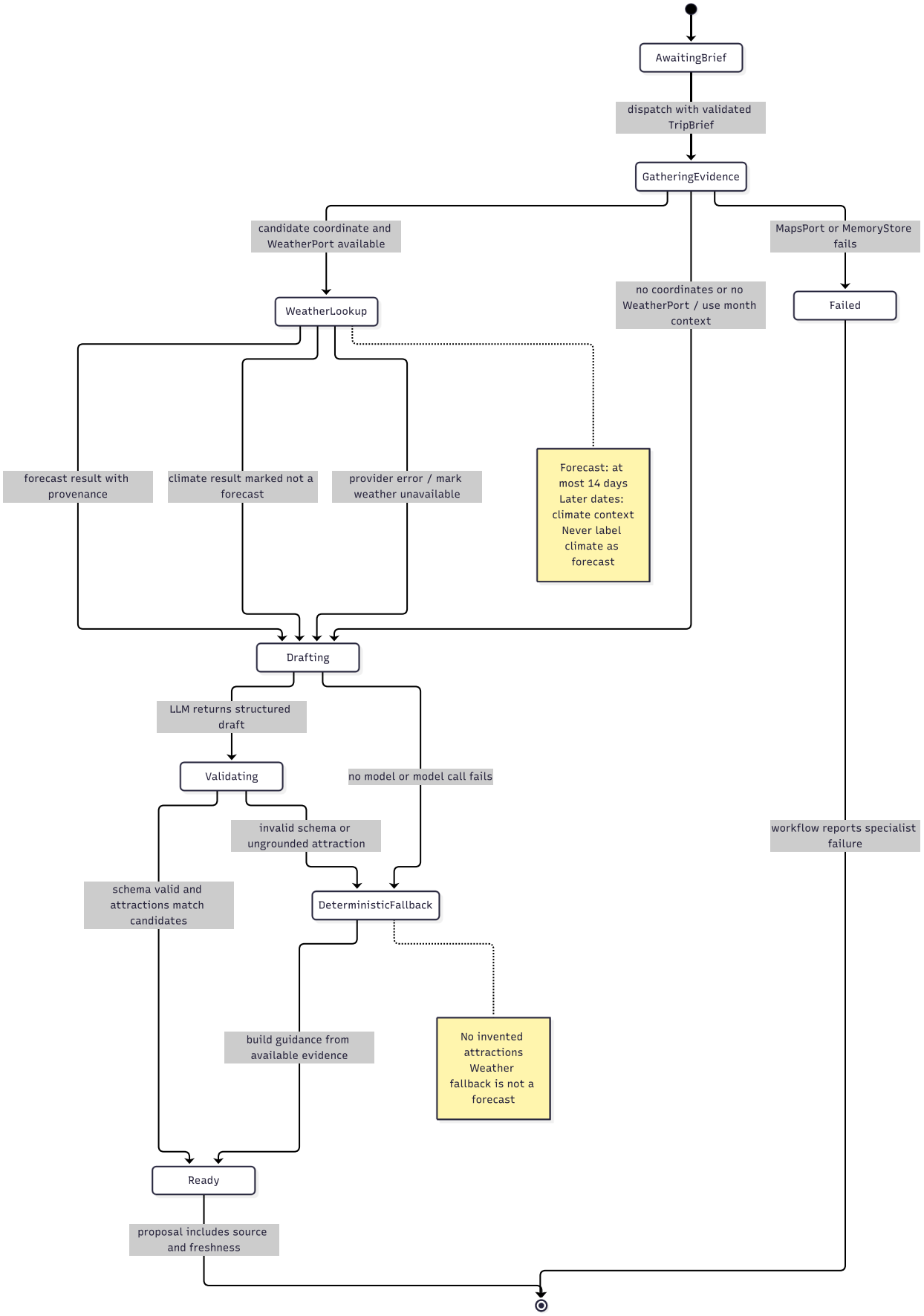
8.4.2 Sequence diagram: weather-based packing and dietary dining (UC-D1)

The LLM receives validated evidence through read-only tools. Provider calls and post-model validation remain deterministic.



8.4.3 State machine: destination-guide weather and packing section

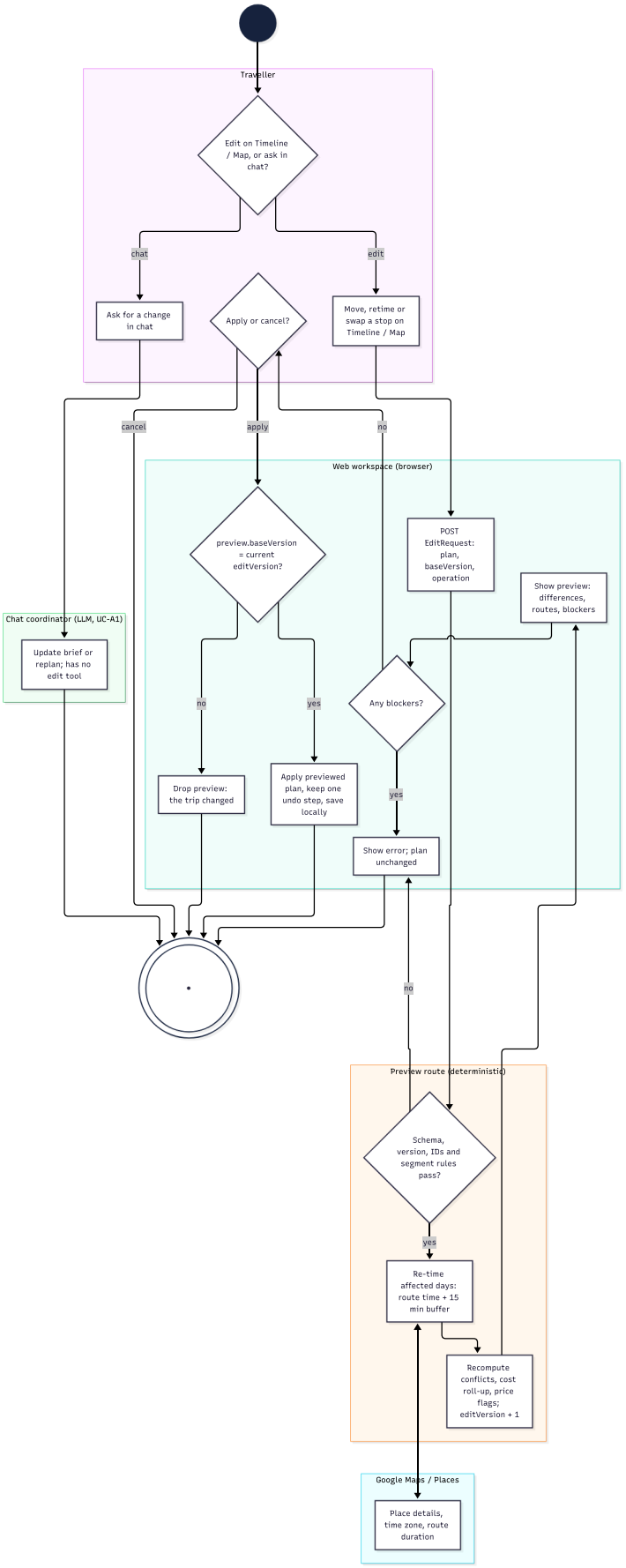
The state machine follows the destination-guide proposal through evidence collection, model validation and fallback. Dining produces a separate proposal in parallel.


AwaitingBrief
dispatch with validated TripBrief
GatheringEvidence
candidate coordinate and WeatherPort available
MapsPort or MemoryStore fails
no coordinates or no WeatherPort / use month context
Failed
WeatherLookup
forecast result with provenance
climate result marked not a forecast
provider error / mark weather unavailable
Forecast: at most 14 days Later dates: climate context Never label climate as forecast
Drafting
LLM returns structured draft
Validating
no model or model call fails
invalid schema or ungrounded attraction
workflow reports specialist failure
schema valid and attractions match candidates
DeterministicFallback
build guidance from available evidence
No invented attractions Weather fallback is not a forecast
Ready
proposal includes source and freshness

8.5 Member E (`@WhW0591`, chat workspace, timeline and map editing)

These models derive from **AH-E1 / R-E1–R-E6** (§2.3.5) and **UC-E1 Edit Itinerary (Timeline / Map)** (§4.2). The chat coordinator can update the brief or replan, but it has no edit tool; the timeline and map edit path is deterministic from preview through local application.

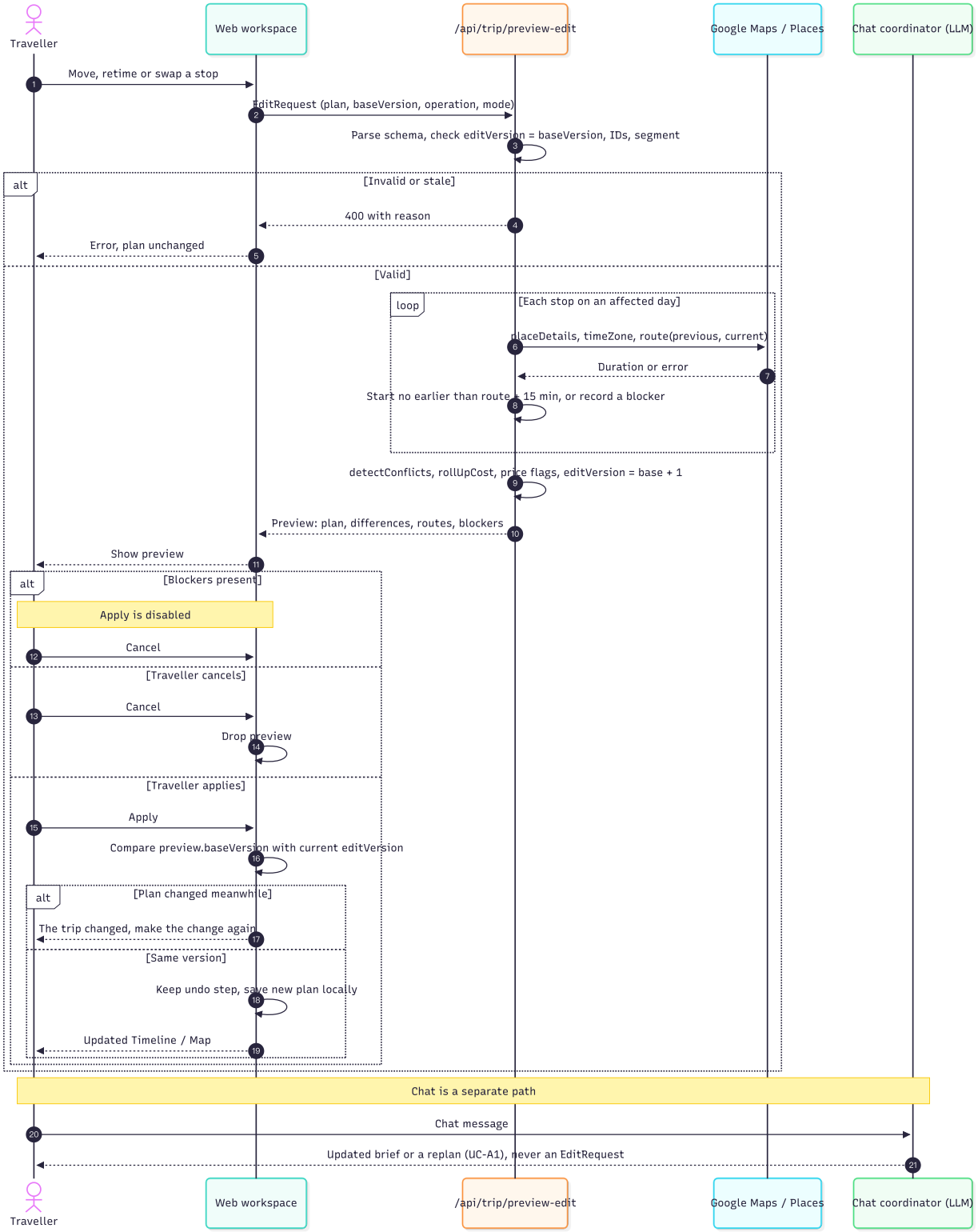
8.5.1 Activity diagram: Edit Itinerary (UC-E1)



The activity diagram separates the current chat path from the implemented Timeline/Map edit path. The chat coordinator can update the brief or replan, but it does not produce an `EditRequest`. The edit path is

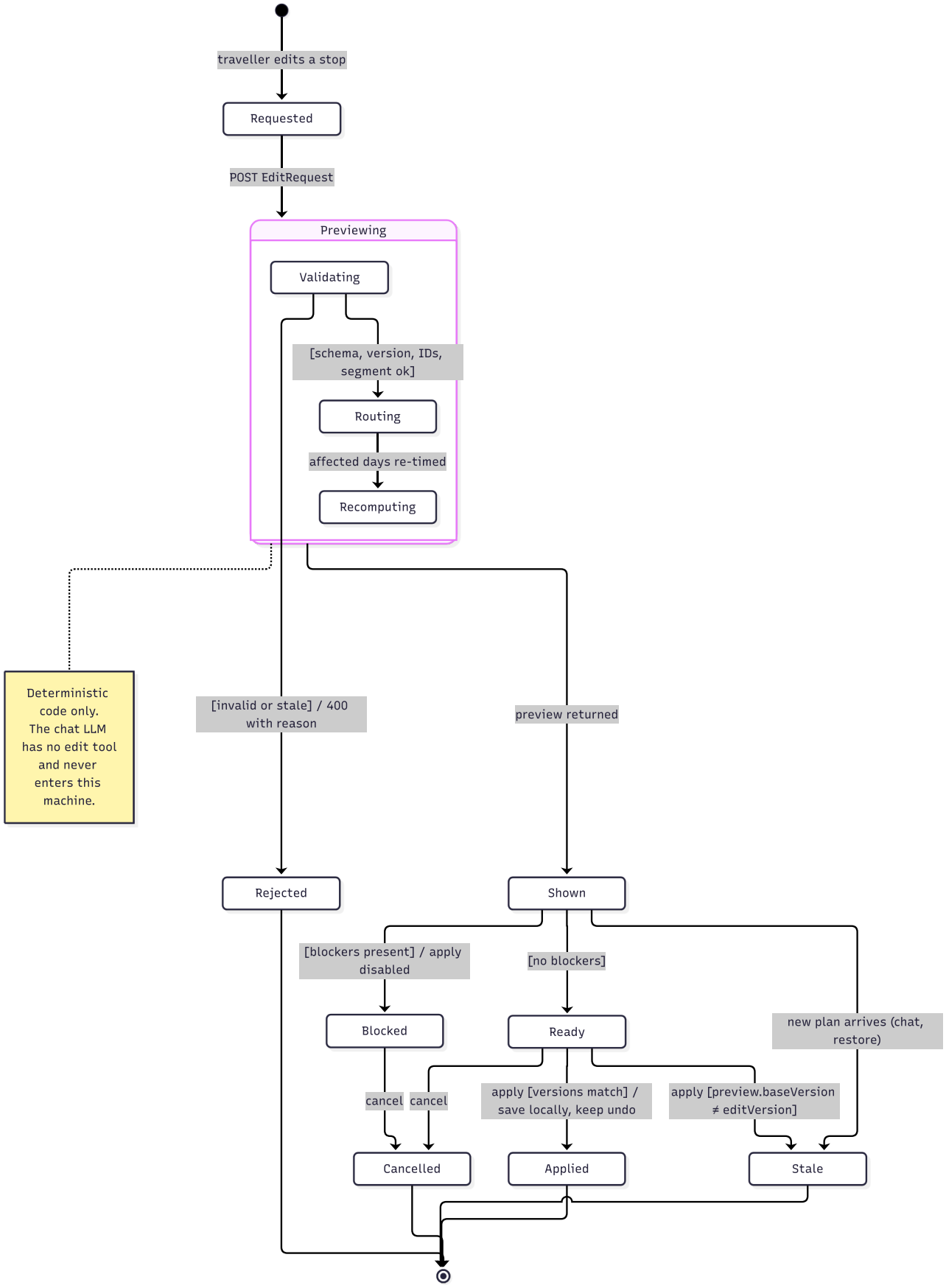
deterministic from preview through local application.

8.5.2 Sequence diagram: Timeline/Map edit with preview and version check (UC-E1)



The sequence diagram shows the implemented Timeline/Map path and marks chat as a separate coordinator path. The important control point is the browser-side comparison of `preview.baseVersion` and the current plan version; persistence is local and there is no server-side atomic commit.

8.5.3 State machine: Timeline/Map edit lifecycle (UC-E1)



The state machine models the implemented Timeline/Map edit lifecycle. It does not include chat interpretation because the current chat coordinator has no edit tool. A request the preview route rejects

ends in `Rejected`. A preview with blockers can only be cancelled. A preview the traveller applies ends in `Applied` when its `baseVersion` still matches the plan, and in `Stale` when the plan changed meanwhile; the applied plan is then saved locally.

9. Relationships between key components and future development

Component	Depends on	Extension point for future work
Shared contracts (`TripBrief`, `AgentProposal`, `TripPlan`, ports)	nothing	A new field is added once, validated by Zod everywhere; a contract change needs an Agent Note, so every owner sees it.
`Specialist` interface and registry	contracts	A sixth specialist (for example, events or visas) implements `invoke` and joins the registry; the graph and board need no change beyond its budget share.
LangGraph workflow	`Specialist`, conflict policy, budget policy	New conflict types plug into `detectConflicts`; the loop, round limit and scoring stay the same.
Planning board	specialist proposals	Stage order and budget shares live in one table, so a new dependency between agents is one row.
`ToolGateway` and ports	provider adapters	A new provider is a new adapter behind an existing port, with mock mode, typed failures and provenance kept.
`MemoryStore` and stores	Redis REST or process memory	Another store realises the same interface without touching agents.
Workspace UI	`ChatResponse`, `TripPlan`, NDJSON events	Reads only contracts, so new sections render through the same card and timeline components.
Agent Lab	the real workflow and specialists	New scenarios and faults are registered values, so each design change can be measured before it ships.

Planned work that these relationships support: verifying live providers end to end, an on-trip mode, flight status through a dedicated provider, and booking links through an affiliate provider.

10. Design rationale and discarded choices

Decision	Chosen	Discarded and why
Who drives the planning loop	A deterministic LangGraph state machine; models act only inside nodes	*Hand-written loop*: revision targeting and round limits were scattered. *Let a model drive the loop*: round limits, conflict detection and cost roll-up must be deterministic and testable.
How agents collaborate	A staged planning board: transport, then accommodation, then itinerary and dining, each with a budget share	*More rounds only*: every round saw the same inputs and blind budget shares overran. *Fully sequential*: destination and dining do not constrain each other, so it only adds latency. *Free model-to-model negotiation*: moves control flow into model output and cannot be tested.
Human in the loop	The traveller edits the plan in chat or the editor; sections still in conflict are marked `needs_you`	*Approval checkpoints* (the early design's "confirm key itinerary"): removed because confirming applied nothing downstream; a real checkpoint needs a booking action the product does not have.
LLM provider	DeepSeek for every model step	*MiniMax*: comparable drafts but 15 to 25 s per structured call; full pages took 37 to 81 s against 15 to 23 s. Kept wired as a backup.
Prices and places	Only from tools, with provenance on every proposal; transport and accommodation wrap deterministic calculators	*Let the model estimate prices*: invents hotels and fares.
Data mode	Mock by default, live chosen per request	*Deploy-time only*: comparing mock and live needed a redeploy. *Per-request environment variable*: leaks between concurrent serverless requests.
Server state	Redis REST store with in-process fallback	*Process memory*: lost on serverless cold starts. *A relational database for everything*: the data is small JSON by key.
Money	AUD in integer cents	Floating-point totals drift.
Evaluation	Agent Lab runs the real workflow against a single-agent baseline	*A separate simpler planner as baseline*: differences would reflect its code, not specialisation.

11. Change assessment and acceptance

Assessing a change. Every proposed change starts as a GitHub issue triaged with one of five labels (`needs-triage`, `needs-info`, `ready-for-agent`, `ready-for-human`, `wontfix`). The owner of the affected package (§5.3) assesses it against three questions:

1. Which requirement (§2) and use case (§4) does it trace to? A change with no trace is either rejected or added to the requirements first.
2. Does it touch a shared contract in `packages/shared`? If so, it needs an Agent Note in the same pull request and every package owner is told, because all packages depend on it.
3. Does it reverse an implemented decision (§10)? If so, a superseding note is written; a decision is never reversed silently.

Accepting a change. A pull request is accepted when:

- CI passes on its head: typecheck, lint, unit and workflow tests, production build, documentation link and pairing checks, and the protected-files check.
- Complex behaviour has an end-to-end test that leaves a repeatable artifact (for agent behaviour, an Agent Lab run artifact).
- The affected use case's postconditions still hold, for example `estTotal` ≤ `budgetTotal` or an explicit `infeasible budget` conflict for UC-C2.
- The documentation and design model describing the change are updated in the same pull request, in English and Chinese.
- The MVP definition of done still holds: grounded and labelled data, visible conflicts after edits, saved trips reopen, and provider failures degrade visibly without fictional prices.

12. Design assumptions

1. One traveller plans one trip at a time; there is no multi-user collaboration.
2. All amounts are stored and compared in AUD. A budget stated in CNY, USD or JPY is converted once, with static rates, when it is read from the traveller's message.
3. Prices are estimates for planning; the system never books or takes payment.
4. Flights are paid first, so accommodation, itinerary and dining share the money left after transport (40%, 40% and 20%).
5. Two guests share a room unless the traveller asks for individual rooms.
6. A model call can fail, time out or return off-schema output at any time, so every model step has a validated deterministic fallback.
7. Providers can be unavailable or over quota; the system then labels estimates or mock data rather than hiding the failure.
8. Three revision rounds are enough for convergence on realistic trips; beyond that the traveller decides.
9. Weather is a forecast only within 14 days; beyond that the guide gives climate context.
10. The traveller can read English or Chinese, and writes dates in any common form.

13. Contributions and use of generative AI

13.1 Individual contributions to group tasks

The group shared every group task equally.

Group task	A	B	C	D	E
Project requirements and classification (§1–2)	20%	20%	20%	20%	20%
Feature diagram (§3)	20%	20%	20%	20%	20%
Overall use case diagram (§4.1)	20%	20%	20%	20%	20%
Architecture, package and deployment (§5)	20%	20%	20%	20%	20%
Class model (§6)	20%	20%	20%	20%	20%
Object diagram, collaboration, structured class (§7)	20%	20%	20%	20%	20%
Relationships, rationale, change, assumptions (§9–12)	20%	20%	20%	20%	20%
Video presentation	20%	20%	20%	20%	20%

13.2 Individual tasks

Member	Ad hoc requirement	Use case specification	Activity	Sequence	State machine
A	AH-A1 (§2.3.1)	UC-A1	§8.1.1	§8.1.2	§8.1.3
B	AH-B1 (§2.3.2)	UC-B1	§8.2.1	§8.2.2	§8.2.3
C	AH-C1 (§2.3.3)	UC-C1, UC-C2	§8.3.1	§8.3.2	§8.3.3
D	AH-D1 (§2.3.4)	UC-D1	§8.4.1	§8.4.2	§8.4.3
E	AH-E1 (§2.3.5)	UC-E1	§8.5.1	§8.5.2	§8.5.3

13.3 Use of generative AI

Parts of this report were drafted with Claude (Anthropic) from the group's repository and design notes, and then reviewed by the group. The models were checked against the code on `main`, and each member can explain their own diagrams.